

SOFTWARE REQUIREMENTS SPECIFICATION

SurplusDrop

Real-time Dutch-auction marketplace for near-expiry surplus inventory

Document	Requirements Analysis · HLD · LLD · Tech Stack · Master Prompt
Version	1.1 (stack update: React + Vite, Express; monorepo; delivery-ready)
Status	Approved for build
Date	23 September 2026
Target	Production-grade PWA, 20-week build, single engineer
Classification	Portfolio project — confidential until public launch

Document control

Item	Value
Owner	Project author (Full-stack engineer)
Audience	The engineer building the system, AI coding assistants, reviewers, interviewers
Normative language	MUST / MUST NOT = mandatory. SHOULD = expected unless a documented reason exists. MAY = optional. (RFC 2119)
Priority scale	P0 = required for MVP (week 12). P1 = required for public launch (week 20). P2 = post-launch stretch.
Change policy	Any change to a MUST requirement, a state machine, or a money flow requires a new Architecture Decision Record (ADR) and a version bump of this document.
Revision history	v1.0 baseline. v1.1 frontend Next.js → React + Vite + React Router; backend NestJS → Express; monorepo with .claude/ tooling; delivery-ready order model; future roadmap (AI assistant, delivery). Companion document: *SurplusDrop AI Engineering Workflow*.
Units	Money is stored and transmitted as integer paise (INR × 100). Time is stored as UTC timestamp _{tz} and displayed in Asia/Kolkata (IST).

Strictness rule. If an implementation detail is not specified here, the engineer chooses the option that (1) never loses or double-counts money, (2) never oversells stock, (3) keeps the server authoritative, in that order. The decision is then recorded as an ADR.

Contents

1. Context and Problem Statement	6
1.1 Background	6
1.2 Solution	6
1.3 Why a Dutch auction	6
1.4 Differentiation	6
1.5 Goals	7
1.6 Non-goals	7
1.7 Success metrics	7
2. Stakeholders, Personas and Roles	8
2.1 Personas	8
2.2 Roles and permissions	8
3. Scope and Release Plan	9
3.1 Explicitly out of scope	9
4. Business Rules (Normative)	10
4.1 Auction configuration	10
4.2 Price function (single source of truth)	10
4.3 Purchase rules	11
4.4 Pickup, cancellation and refund policy	11
4.5 Fees, commission and payout	12
4.6 Seller compliance	12
5. Functional Requirements	13

5.1 Identity and access (AUTH)	13
5.2 Seller onboarding and store management (STORE)	13
5.3 Auction lots (LOT)	14
5.4 Discovery (DISC)	14
5.5 Orders and payments (ORD / PAY)	15
5.6 Pickup (PICK)	15
5.7 Notifications (NOTIF)	16
5.8 Trust and safety (TRUST)	16
5.9 Seller analytics (ANLT)	16
5.10 AI Price Advisor (AI)	17
5.11 Administration (ADM)	17
6. Non-Functional Requirements	18
6.1 Performance and capacity	18
6.2 Reliability and data integrity	18
6.3 Scalability	18
6.4 Security	19
6.5 Usability, accessibility and compatibility	19
6.6 Observability	19
7. Edge-Case Catalogue (Strict)	20
7.1 Concurrency and stock	20
7.2 Time and clocks	20
7.3 Payments	20
7.4 Realtime and network	21
7.5 Pickup, abuse and data	21
8. Technology Stack	22
8.1 Frontend	22
8.2 Backend	22
8.3 Quality, delivery and operations	23
9. High-Level Design (HLD)	24
9.1 Architecture style	24
9.2 Components	24
9.3 Module map	25
9.4 Critical flow — reserve and pay	25
9.5 Realtime design	26
9.6 Consistency and failure strategy	27
9.7 Caching strategy	27
9.8 Scaling plan and known bottlenecks	28
9.9 Deployment topology	28

9.10 Architecture Decision Records (summary)	28
10. Low-Level Design (LLD)	30
10.1 Repository structure	30
10.2 Layering rules inside a module	30
10.3 Database schema (core DDL)	31
10.4 State machines	34
10.5 Core algorithms	35
10.6 REST API specification	38
10.7 Socket.io event contract	39
10.8 Redis key design	40
10.9 Queues and jobs	40
10.10 Configuration (environment variables)	41
11. UI / UX Specification	42
11.1 Design principles	42
11.2 Design tokens	42
11.3 Signature components	42
11.4 Screen inventory	42
11.5 Responsive layout rules	43
11.6 Interaction rules (strict)	43
12. Quality: Testing, Load and Observability	44
12.1 Test strategy	44
12.2 Load and resilience test plan	44
12.3 Dashboards	45
12.4 Alerts	45
12.5 Runbooks (in docs/runbooks)	45
13. Delivery, DevOps and Release	46
13.1 Source control and reviews	46
13.2 CI/CD pipeline	46
13.3 Database migrations	46
13.4 Environments and data	46
13.5 Definition of Done (every feature)	46
13.6 20-week delivery plan	47
14. Master Prompt for AI-Assisted Development	48
14.1 Per-task prompt template	50
15. Future Roadmap (Post-Launch Add-ons)	51
15.1 Delivery-ready design (applies now)	51
15.2 Add-on 1 — AI assistant for buyers and sellers	51
15.3 Add-on 2 — Delivery	51

16. Appendices 52

16.1 Appendix A — Glossary 52

16.2 Appendix B — Price Advisor heuristic (deterministic) 52

16.3 Appendix C — Launch checklist 52

16.4 Appendix D — Evidence to show recruiters 53

1. Context and Problem Statement

1.1 Background

Bakeries, restaurants, cafés, sweet shops, grocery stores and cloud kitchens produce stock that loses all value within hours: bread baked in the morning, prepared meals at closing time, fruit one day from spoiling, dairy on its best-before date. Today this stock is either discarded, given away informally, or discounted by hand with a paper sign that only walk-in customers see. The seller recovers little or nothing, and edible food becomes waste.

Existing surplus-food apps sell **fixed-price "surprise bags"**. The seller must guess a single price in advance; if it is too high the bag does not sell, if it is too low the seller loses money. The buyer does not know what is inside. Neither side gets real-time price discovery.

1.2 Solution

SurplusDrop lets a seller list a specific item (for example "12 × multigrain loaf, best before tonight") as a **live Dutch auction**: the price starts high and drops by a fixed step at a fixed interval until it reaches a floor price or the stock sells out. Buyers within a few kilometres see the price falling in real time on a map and choose when to buy. Whoever buys first at the current price gets the unit; the buyer pays online and collects the item in person during a pickup window using a one-time code.

1.3 Why a Dutch auction

- **Price discovery**: the market, not the seller, finds the price each buyer is willing to pay. Early, eager buyers pay more; bargain hunters wait and risk the item selling out.
- **Speed**: a Dutch auction ends at the first acceptance of each unit, so it suits perishable goods where time is the constraint.
- **Engagement**: a falling price creates urgency and a reason to keep the app open, which is exactly the real-time behaviour this project must demonstrate.
- **Fairness and simplicity**: there is no bidding war and no bid history to store; every buyer pays the price shown at the moment they commit.

1.4 Differentiation

Dimension	Surprise-bag apps	Manual discount sign	SurplusDrop
Pricing	Fixed, guessed in advance	Fixed, changed by hand	Time-decaying, server-computed, AI-advised
What the buyer sees	Mystery contents	Only if walking past	Exact item, photo, quantity, live price
Reach	App users, static list	Walk-in only	Push + live map to buyers within radius
Seller insight	Minimal	None	Sell-through, price-at-sale, time-to-sellout, waste saved
Real-time	No	No	Stock and state changes pushed within 500 ms

1.5 Goals

- G1. Let a verified seller create a live auction in under 60 seconds from a phone.
- G2. Let a buyer discover, buy and pay for a nearby item in under 30 seconds.
- G3. **Never oversell** a unit and **never lose or double-charge** a payment, under any concurrency or failure.
- G4. Push every stock and state change to every interested client within 500 ms (p95).
- G5. Operate as a production system: observable, horizontally scalable, deployable by CI, recoverable from backup.

1.6 Non-goals

- Delivery or logistics. Pickup only.
- Native iOS/Android apps (the PWA is the mobile product; see ADR-006).
- Buyer-seller chat, loyalty points, wallets or credits (v2 candidates).
- Multi-currency or international sellers. INR and India only.
- Ascending (English) auctions or bidding of any kind.

1.7 Success metrics

Metric	Definition	Target
Oversell count	Confirmed units – total units, per lot	Exactly 0 , always (hard invariant, alerted)
Payment integrity	Captured payments without a CONFIRMED order or a refund, older than 30 min	0
Realtime latency	DB commit → client receives event	p95 ≤ 500 ms
Reservation latency	POST /orders server time	p95 ≤ 250 ms at 500 req/s
Sell-through rate	Units sold / units listed, per seller per week	≥ 60% (product health)
Checkout conversion	CONFIRMED / PENDING_PAYMENT orders	≥ 70%
Availability	Successful health checks per month	≥ 99.5%

2. Stakeholders, Personas and Roles

2.1 Personas

Persona	Profile	Needs	Frustrations
Buyer — "Riya"	Student / young professional, 19–32, phone-first, price-sensitive, lives within 3 km of many food outlets	Cheap, good food nearby; know exactly what she gets; fast UPI checkout	Mystery bags; items gone on arrival; slow checkout
Seller — "Mr. Arora"	Owner of a bakery/café, uses a phone at the counter and a laptop at night	Recover cost on unsold stock with near-zero effort; see what works	Complex dashboards; setting prices; no-show customers
Seller staff	Counter staff handing over orders	Verify pickup in 2 taps, even with bad internet	Typing long codes; confusion over who paid
Admin / Ops	Platform operator (the author)	Approve sellers, resolve disputes, see payment health, act fast in incidents	Missing audit trail; manual reconciliation

2.2 Roles and permissions

A single `users` account MAY hold several roles. A user becomes a seller by creating a store, which then requires admin approval. Store access is granted through `store_members` with a store-level role (OWNER or STAFF).

Capability	Guest	Buyer	Seller STAFF	Seller OWNER	Admin
Browse nearby live lots, view lot detail	✓	✓	✓	✓	✓
Reserve and pay for a lot	—	✓	✓ (not own store)	✓ (not own store)	—
Follow stores, saved alerts, push opt-in	—	✓	✓	✓	—
Rate an order, raise a dispute	—	✓	—	—	—
Verify pickup code	—	—	✓	✓	—
Create / publish / cancel lots	—	—	—	✓	—
View store analytics and payouts	—	—	—	✓	✓
Invite / remove store staff	—	—	—	✓	—
Approve / suspend stores and users	—	—	—	—	✓
Resolve disputes, issue manual refunds	—	—	—	—	✓
Toggle feature flags, view audit log	—	—	—	—	✓

A seller MUST NOT be able to buy from their own store (prevents fake sell-through and self-dealing). Admins MUST NOT place orders from an admin session.

3. Scope and Release Plan

Module	P0 — MVP (wk 12)	P1 — Launch (wk 20)	P2 — Stretch
Identity	Email + password, email OTP verification, JWT + refresh rotation	Google OAuth, session list + remote logout	Phone OTP login
Seller onboarding	Store profile, map pin, hours, FSSAI number, admin approval	Staff invites, store documents	Razorpay Route KYC + automated payouts
Auctions	Create, schedule, go live, decay, sold out, end, cancel	Duplicate lot, templates, end early	Recurring daily lots
Discovery	Map + list, radius, category, sort by ending soon / price / distance	Search by text, filters by max price and discount %	Personalised feed
Realtime	Geo rooms, lot rooms, user rooms, stock + state events, time sync	Presence ("14 people watching")	—
Orders & payments	Reserve, Razorpay checkout, webhook confirmation, expiry, late capture, refunds	Reconciliation job + admin report	Partial refunds by seller
Pickup	6-digit code + signed QR, seller verify, no-show automation	Offline-tolerant verify (queued)	—
Notifications	In-app notification centre, order emails	Web Push, follows, saved alerts, quiet hours	WhatsApp
Trust & safety	Ratings, disputes (admin resolves), seller strikes	Buyer reservation cooldown, abuse rate limits	Fraud scoring
Analytics	Per-lot results	Seller dashboard: revenue, sell-through, price-at-sale chart, waste saved	Exports (CSV)
AI	—	Price Advisor (heuristic + LLM explanation), listing text improver	Photo → listing draft (vision); seller MCP server
Admin	Store approval, user suspend, dispute resolution, audit log	Payments dashboard, feature flags	—

3.1 Explicitly out of scope

- Delivery, rider assignment, live tracking of couriers (planned add-on; the data model is delivery-ready — see Future Roadmap).
- Cash on pickup. Every order is prepaid online (removes no-show loss for sellers).
- Selling alcohol, tobacco, medicines, or any item requiring age or prescription checks. Category allow-list enforced server-side.
- Tipping, coupons, referral credits, subscriptions.

4. Business Rules (Normative)

These rules are the contract between product and code. Each rule has an ID and **MUST** be enforced **on the server** and covered by at least one automated test that references the ID in its name (for example `it("BR-AUC-04 rejects floor ≥ start")`). Client-side checks exist only for user experience.

4.1 Auction configuration

ID	Rule
BR-AUC-01	All prices are integers in paise. <code>start_price</code> ∈ [1 000, 1 000 000] (₹10 to ₹10 000).
BR-AUC-02	<code>floor_price</code> ≥ 100 (₹1) and <code>floor_price</code> ≤ 0.9 × <code>start_price</code> (at least 10% total decay).
BR-AUC-03	<code>mrp</code> is optional; if present <code>mrp</code> ≥ <code>start_price</code> . Discount shown to buyers = 1 – <code>current_price</code> / <code>mrp</code> .
BR-AUC-04	<code>decrement_amount</code> ≥ 100 paise and ≤ (<code>start_price</code> – <code>floor_price</code>).
BR-AUC-05	<code>decrement_interval_sec</code> ∈ {60, 120, 300, 600, 900}. No other values accepted.
BR-AUC-06	Duration (<code>ends_at</code> – <code>starts_at</code>) ∈ [15 min, 12 h]. <code>starts_at</code> ∈ [now, now + 7 days]. A <code>starts_at</code> in the past within 60 s is snapped to now.
BR-AUC-07	<code>qty_total</code> ∈ [1, 500]. <code>max_qty_per_order</code> ∈ [1, 10], default 5, and ≤ <code>qty_total</code> .
BR-AUC-08	Pickup window: <code>pickup_start</code> ≥ <code>starts_at</code> ; <code>pickup_end</code> ≥ <code>ends_at</code> + 30 min; <code>pickup_end</code> ≤ <code>ends_at</code> + 12 h; <code>pickup_end</code> – <code>pickup_start</code> ≥ 30 min.
BR-AUC-09	Food categories require <code>best_before</code> and <code>best_before</code> ≥ <code>pickup_end</code> . Items past best-before MUST NOT be listed.
BR-AUC-10	Once a lot is LIVE, the only allowed edits are: increase <code>qty_total</code> , end early, cancel. Prices, times and description are frozen (fairness to buyers who are watching).
BR-AUC-11	A store MAY have at most 20 lots in SCHEDULED or LIVE at once.
BR-AUC-12	Only stores with status APPROVED and not SUSPENDED can publish. A store suspension cancels its SCHEDULED lots and ends its LIVE lots (existing CONFIRMED orders are honoured).
BR-AUC-13	Allowed categories: BAKERY, MEALS, SWEETS, FRUITS_VEG, DAIRY, GROCERY, BEVERAGES, OTHER_FOOD. Anything else is rejected.

4.2 Price function (single source of truth)

The current price is a **pure, deterministic function** of the lot configuration and time. It lives in `packages/shared/src/pricing.ts`, is imported by both server and client, and has 100% branch coverage plus property-based tests (fast-check).

packages/shared/src/pricing.ts

```
export function priceAt(lot: PriceConfig, nowMs: number): number {
  if (nowMs < lot.startsAtMs) return lot.startPrice; // not started: show start price
  const elapsed = Math.min(nowMs, lot.endsAtMs) - lot.startsAtMs;
  const steps = Math.floor(elapsed / (lot.decrementIntervalSec * 1000));
  return Math.max(lot.floorPrice, lot.startPrice - steps * lot.decrementAmount);
}

export function nextDropAt(lot: PriceConfig, nowMs: number): number | null {
  if (priceAt(lot, nowMs) === lot.floorPrice) return null; // already at floor
  const i = lot.decrementIntervalSec * 1000;
  const k = Math.floor((nowMs - lot.startsAtMs) / i) + 1;
  const t = lot.startsAtMs + k * i;
  return t < lot.endsAtMs ? t : null;
}

// Properties that MUST hold (property tests):
// P1 monotonic: t1 ≤ t2 => priceAt(t1) ≥ priceAt(t2)
// P2 bounded: floorPrice ≤ priceAt(t) ≤ startPrice
// P3 integer: Number.isInteger(priceAt(t))
```

The server never trusts the client clock. At reservation time the price is computed from the database clock (`now()` inside the transaction). The client clock is used only for the countdown animation, corrected by a measured server-time offset (see Realtime design).

4.3 Purchase rules

ID	Rule
BR-BUY-01	A reservation succeeds only if the lot is LIVE, <code>now()</code> < <code>ends_at</code> , and <code>qty_available</code> ≥ <code>qty</code> — checked and decremented in one atomic statement.
BR-BUY-02	Unit price is locked at reservation time. <code>Total</code> = <code>unit_price</code> × <code>qty</code> . There are no buyer fees in v1.
BR-BUY-03	The client sends <code>expectedUnitPrice</code> . If the server price is higher (client clock skew), reject with <code>PRICE_MISMATCH</code> and return the current price. If lower, accept at the lower price.
BR-BUY-04	Payment hold: an order in <code>PENDING_PAYMENT</code> holds its units for 5 minutes (<code>hold_expires_at</code>). After that the units return to the lot.
BR-BUY-05	A buyer MAY buy at most <code>max_qty_per_order</code> units per lot in total, across all non-expired, non-cancelled orders for that lot.
BR-BUY-06	A buyer MAY have at most 2 orders in <code>PENDING_PAYMENT</code> at once across the platform.
BR-BUY-07	Cooldown: 3 orders that EXPIRED within a rolling 24 h block new reservations for that buyer for 24 h (<code>RESERVATION_COOLDOWN</code>).
BR-BUY-08	A buyer MUST have a verified email before reserving.
BR-BUY-09	Every POST that creates or changes money state MUST carry an Idempotency-Key header (UUID v4). Same key + same body within 24 h returns the stored response; same key + different body returns <code>IDEMPOTENCY_KEY_REUSED</code> .

4.4 Pickup, cancellation and refund policy

Situation	Order result	Money result	Other effect
Buyer cancels before paying	<code>CANCELLED_BY_BUYER</code>	Nothing captured	Units returned immediately
Hold expires unpaid	<code>EXPIRED</code>	Nothing captured	Units returned; counts toward BR-BUY-07
Payment captured after expiry, stock still available	<code>EXPIRED</code> → <code>CONFIRMED</code>	Kept	Units re-reserved at the original unit price
Payment captured after expiry, no stock	Stays <code>EXPIRED</code>	Automatic full refund	Buyer notified with apology
Buyer wants to cancel after paying	Not allowed in v1	No refund	UI states this before checkout (explicit consent checkbox)
Seller cancels a CONFIRMED order or the lot	<code>CANCELLED_BY_SELLER</code>	Full refund	Seller receives 1 strike
Buyer does not arrive by pickup_end + 30 min	<code>NO_SHOW</code>	Seller is paid	No refund
Buyer arrives, store closed / item missing	Dispute (≤ 24 h after <code>pickup_end</code>)	Admin decides: full, partial or none	Upheld dispute = 1 seller strike
Seller gets 3 strikes in 30 days	—	—	Store auto-SUSPENDED, admin review required

4.5 Fees, commission and payout

- Platform commission: **10%** of order total, deducted from the seller side. Stored as `commission_bps = 1000` per store (admin-editable, never retroactive).
- Payment gateway fees are recorded from the Razorpay payment entity (`fee`, `tax`) into the ledger; in v1 the platform absorbs them.
- Seller payable = total – commission. It becomes **setttableable** when the order reaches `PICKED_UP` or `NO_SHOW` and the dispute window has closed.
- P0/P1: payouts are settled manually by admin from a generated payout report (ledger-backed). P2: Razorpay Route linked accounts with automated transfers.

4.6 Seller compliance

- Stores selling food **MUST** provide a 14-digit FSSAI licence/registration number (format validated; not verified against the government registry in v1 — shown as "self-declared").
- GSTIN is optional; if provided it **MUST** match the 15-character GSTIN pattern.
- Personal data handling follows India's Digital Personal Data Protection Act, 2023: purpose-limited collection, consent at signup, account deletion on request, no sale of data.

5. Functional Requirements

Each requirement is testable. "Acceptance criteria" describes the observable behaviour an automated or manual test MUST verify.

5.1 Identity and access (AUTH)

ID	Requirement	Pri	Acceptance criteria
FR-AUTH-01	Sign up with name, email, password. Password $\geq$ 10 chars, checked against a breached-password list (top 100k). Hashed with Argon2id.	P0	Weak/breached password rejected with field error; DB stores only Argon2id hash.
FR-AUTH-02	Email verification via 6-digit OTP, valid 10 min, max 5 attempts, resend after 60 s, max 5 sends/day.	P0	6th wrong attempt locks OTP; resend within 60 s returns 429.
FR-AUTH-03	Login returns a 15-min access JWT and a 30-day refresh token, both in HttpOnly; Secure; SameSite=Lax cookies.	P0	Tokens not readable from JS; access expires at 15 min.
FR-AUTH-04	Refresh token rotation with reuse detection: reusing an old refresh token revokes the whole token family.	P0	Replay of a rotated token $\rightarrow$ 401 and all sessions in that family revoked.
FR-AUTH-05	Login rate limit: 5 failures per 15 min per (email, IP) $\rightarrow$ temporary lock with generic error.	P0	No user-enumeration: same message for unknown email and wrong password.
FR-AUTH-06	Password reset via emailed single-use link (30 min, hashed token in DB). Reset revokes all sessions.	P0	Old sessions return 401 after reset.
FR-AUTH-07	Google OAuth (OIDC) login; links to existing account only if the Google email is verified and matches.	P1	Unverified Google email cannot take over an existing account.
FR-AUTH-08	Active sessions list with device/browser, last seen, and "log out other sessions".	P1	Remote logout invalidates refresh token within 1 request.
FR-AUTH-09	Account deletion: soft-delete immediately, anonymise PII after 30 days; financial records retained (legal) with PII stripped.	P1	Deleted user cannot log in; orders show "Deleted user".

5.2 Seller onboarding and store management (STORE)

ID	Requirement	Pri	Acceptance criteria
FR-STORE-01	Create store: name, category, description, phone, address, map pin (lat/lng), opening hours per weekday, FSSAI number, optional GSTIN, logo, cover image.	P0	Missing FSSAI for food store $\rightarrow$ 422 with field error.
FR-STORE-02	Store status machine: PENDING_REVIEW $\rightarrow$ APPROVED REJECTED; APPROVED $\nrightarrow$ SUSPENDED. Only admin changes status, with reason.	P0	Unapproved store cannot publish lots (403 STORE_NOT_APPROVED).
FR-STORE-03	Map pin MUST be within India bounding box and within 300 m of the geocoded address; otherwise require confirmation.	P0	Pin in the sea $\rightarrow$ validation error.
FR-STORE-04	Owner invites staff by email; staff role can only verify pickups and view today's orders.	P1	Staff calling create-lot API $\rightarrow$ 403.
FR-STORE-05	Holiday mode: store hidden from discovery and cannot publish; scheduled lots stay scheduled but blocked from going live.	P1	Scheduler skips lots of stores in holiday mode and marks them CANCELLED with reason.

5.3 Auction lots (LOT)

ID	Requirement	Pri	Acceptance criteria
FR-L0T-01	Create lot (DRAFT): title (5-80 chars), description (≤ 500), category, 1-4 images, qty, max per order, MRP, start price, floor, decrement amount/interval, starts_at, ends_at, pickup window, best_before, dietary tags (VEG, NON_VEG, VEGAN, CONTAINS_NUTS, CONTAINS_GLUTEN).	P0	All BR-AUC rules enforced with per-field errors.
FR-L0T-02	Live preview of the price curve (step chart) and "reaches floor at HH:MM" while editing.	P0	Chart uses shared priceAt; matches server.
FR-L0T-03	Publish: DRAFT $\rightarrow$ SCHEDULED. Scheduler moves SCHEDULED $\rightarrow$ LIVE at starts_at (± 2 s) and LIVE/SOLD_OUT $\rightarrow$ ENDED at ends_at (± 2 s).	P0	Lot scheduled for T is LIVE at T+2s latest in integration test with fake timers.
FR-L0T-04	Images uploaded directly to object storage via presigned PUT; ≤ 5 MB, JPEG/PNG/WebP only, re-encoded to WebP 1200 px + 400 px thumbnail by a worker, EXIF stripped.	P0	Upload of a .svg or a 6 MB file rejected; stored images have no EXIF GPS.
FR-L0T-05	End early (LIVE $\rightarrow$ ENDED) and cancel (any non-terminal $\rightarrow$ CANCELLED). Cancel with CONFIRMED orders requires typed confirmation and triggers refunds (BR policy).	P0	Cancel refunds every CONFIRMED order exactly once.
FR-L0T-06	Increase quantity while LIVE; SOLD_OUT lot returns to LIVE if before ends_at.	P1	Buyers receive lot.stock_changed with new qty.
FR-L0T-07	Duplicate a past lot into a new DRAFT with times shifted to today.	P1	Duplicate copies config, not images of deleted storage keys.

5.4 Discovery (DISC)

ID	Requirement	Pri	Acceptance criteria
FR-DISC-01	Nearby lots: given lat/lng and radius (1, 2, 3, 5, 10 km; default 3), return LIVE and SCHEDULED (starting ≤ 2 h) lots from APPROVED stores, with distance.	P0	PostGIS ST_DWithin on geography; p95 ≤ 120 ms with 10k live lots.
FR-DISC-02	Sort: ending soon (default), lowest current price, biggest discount, nearest. Current price sorting is computed server-side at request time.	P0	Sort by price uses priceAt at request time, not stored price.
FR-DISC-03	Filters: category, dietary tags, max current price, "starting soon", minimum discount %.	P1	Combined filters are AND-ed; empty result shows empty state.
FR-DISC-04	Map view (clustered markers) and list view, synced; list is cursor-paginated (20 per page).	P0	No duplicates or gaps across pages while stock changes.
FR-DISC-05	Location: browser geolocation with permission; fallback to manual pick on map or city search. Location is never stored server-side beyond the request (privacy).	P0	Denied permission still allows manual location.
FR-DISC-06	Lot detail page: images, store info, rating, distance, live price, next-drop countdown, price curve, qty left, pickup window, dietary tags, best-before.	P0	Shared links show photo, title and price via the server share route; page updates live.
FR-DISC-07	Text search on title/store name (Postgres full-text, simple config + trigram for typos).	P1	"multigran" finds "multigrain".

5.5 Orders and payments (ORD / PAY)

ID	Requirement	Pri	Acceptance criteria
FR-ORD-01	Reserve: POST /orders {lotId, qty, expectedUnitPrice} with Idempotency-Key. Creates PENDING_PAYMENT order and Razorpay order; returns checkout params and holdExpiresAt.	P0	Under 2 000 concurrent requests for 50 units, exactly 50 units reserved.
FR-ORD-02	Checkout screen shows a hold countdown; at 0 it disables pay and offers "try again" (new reservation).	P0	Client never submits payment for an expired hold.
FR-ORD-03	Payment confirmation source of truth is the webhook payment.captured (auto-capture enabled). Client callback only triggers a signature check and a fast UI update.	P0	Order confirms even if the browser is closed after paying.
FR-ORD-04	On confirmation: generate 6-digit pickup code and signed QR, send receipt email, push + in-app notification.	P0	Code unique among CONFIRMED orders of the same store.
FR-ORD-05	Buyer can cancel a PENDING_PAYMENT order (releases units immediately).	P0	Stock event emitted within 500 ms.
FR-ORD-06	My Orders: active (with code/QR and pickup window, directions link) and past orders.	P0	Code visible only to the buyer.
FR-PAY-01	Webhook endpoint verifies X-Razorpay-Signature (HMAC-SHA256 of raw body), stores the event with unique event id, returns 200 within 200 ms, processes asynchronously.	P0	Duplicate delivery processed once; invalid signature → 400 and alert.
FR-PAY-02	Handle events: payment.captured, payment.failed, refund.processed, refund.failed, order.paid.	P0	Unknown event types stored and ignored.
FR-PAY-03	Refunds created via API with our refund_id in notes; idempotent retry on failure (exponential backoff, max 8 attempts, then admin alert).	P0	Never two refunds for the same order and reason.
FR-PAY-04	Reconciliation job every 5 min: for PENDING_PAYMENT/EXPIRED orders with a Razorpay order in the last 48 h, fetch payments from Razorpay and apply missed captures.	P1	Simulated lost webhook is recovered within 10 min.
FR-PAY-05	Double-entry ledger entries for every capture, commission, gateway fee, refund and payout; sum of each transaction = 0.	P0	DB constraint/trigger rejects unbalanced transaction.

5.6 Pickup (PICK)

ID	Requirement	Pri	Acceptance criteria
FR-PICK-01	Seller/staff verifies pickup by entering the 6-digit code or scanning the QR. Allowed from pickup_start – 15 min to pickup_end + 30 min.	P0	Verification outside window returns PICKUP_WINDOW_CLOSED.
FR-PICK-02	QR payload = orderId.code.hmac (HMAC-SHA256 with server secret, truncated 16 bytes, base64url).	P0	Tampered QR rejected; QR of another store rejected.
FR-PICK-03	Code brute-force protection: 5 failed attempts per store per 10 min → 10-min lock for that device/session.	P0	6th attempt returns 429.
FR-PICK-04	Verification is idempotent: verifying an already PICKED_UP order returns success with alreadyVerified: true and the original time.	P0	Double scan does not error at the counter.
FR-PICK-05	No-show job marks CONFIRMED orders NO_SHOW at pickup_end + 30 min and notifies buyer.	P0	Runs even after worker restart (sweeper).
FR-PICK-06	Offline verify: if the seller device is offline, a verified QR is queued locally and synced; server re-validates and resolves conflicts (first verification wins).	P1	Two devices verifying the same order offline → one success, one alreadyVerified.

5.7 Notifications (NOTIF)

ID	Requirement	Pri	Acceptance criteria
FR-NOTIF-01	In-app notification centre with unread count (live via user room).	P0	Unread count consistent across two tabs.
FR-NOTIF-02	Transactional email: OTP, order confirmation (with code), refund issued, dispute outcome, store approval.	P0	Emails sent via queue; retried; never block the API.
FR-NOTIF-03	Web Push (VAPID) opt-in after first order or first follow — never on first page load.	P1	Expired subscriptions (410) deleted automatically.
FR-NOTIF-04	Follow a store → push when it publishes a lot. Saved alert: category + radius + max price → push on match. Max 5 saved alerts per user.	P1	Matching runs in worker; one push per (user, lot).
FR-NOTIF-05	Quiet hours 22:00–08:00 IST for marketing pushes (follows/alerts). Transactional pushes always delivered.	P1	Alert match at 23:00 is dropped, not delayed.
FR-NOTIF-06	Frequency cap: max 6 marketing pushes per user per day.	P1	7th is suppressed and logged.

5.8 Trust and safety (TRUST)

ID	Requirement	Pri	Acceptance criteria
FR-TRUST-01	Buyer rates a PICKED_UP order 1–5 stars with optional comment (≤ 300 chars) within 7 days; once per order.	P0	Store rating = Bayesian average (prior 4.0, weight 5).
FR-TRUST-02	Dispute: reason (ITEM_NOT_AVAILABLE, STORE_CLOSED, QUALITY_ISSUE, WRONG_ITEM), description, up to 3 photos, within 24 h of pickup_end. One dispute per order.	P0	Dispute after window → 422 DISPUTE_WINDOW_CLOSED.
FR-TRUST-03	Admin resolves dispute: FULL_REFUND, PARTIAL_REFUND (amount), REJECTED, with note. Seller can add a response within 24 h.	P0	Resolution creates refund + ledger entries atomically.
FR-TRUST-04	Strike system and auto-suspension (see policy table). Strikes expire after 30 days.	P0	3rd strike suspends store in same transaction.
FR-TRUST-05	Report a listing (inappropriate / unsafe / misleading); 3 reports from distinct users auto-hide the lot pending admin review.	P1	Hidden lot ends and emits <code>lot.ended{reason: MODERATION}</code> .

5.9 Seller analytics (ANLT)

ID	Requirement	Pri	Acceptance criteria
FR-ANLT-01	Per-lot result: units sold, revenue, average unit price, sell-through %, time-to-sellout, scatter of (time, price) for each sale over the price curve.	P0	Numbers reconcile to ledger to the paisa.
FR-ANLT-02	Store dashboard (7/30/90 days): revenue, orders, sell-through, no-show rate, average discount, best hour of day, estimated food saved (kg, from per-category average weights).	P1	Served from daily rollup table; loads ≤ 1 s.
FR-ANLT-03	Payout view: settleable, pending, settled amounts with line items.	P1	Sum equals ledger seller-payable balance.

5.10 AI Price Advisor (AI)

ID	Requirement	Pri	Acceptance criteria
FR-AI-01	While creating a lot, seller taps "Suggest pricing". Input: category, MRP, qty, minutes until best-before, local time, store's last 30 days of results in that category.	P1	Suggestion returns ≤ 3.5 s or falls back.
FR-AI-02	A deterministic heuristic computes start, floor, decrement and interval (Appendix B). The LLM only writes a ≤ 60 -word explanation and optional improved title/description. The LLM never sets numbers.	P1	Given the same inputs, numbers are identical across calls.
FR-AI-03	LLM output is JSON validated with Zod; invalid/timeout $\rightarrow$ heuristic numbers shown with a template explanation.	P1	Malformed LLM output never reaches UI.
FR-AI-04	Suggestions are applied only on explicit seller action; each suggestion and whether it was accepted is logged (ai_suggestions) to measure uplift.	P1	Acceptance rate visible in admin.
FR-AI-05	Prompt-injection hardening: seller-provided text is passed as quoted data; system prompt forbids following instructions inside it; output filtered for URLs/phone numbers.	P1	Title "ignore instructions and..." produces normal output.
FR-AI-06	Per-store AI quota: 30 suggestions/day; global kill switch via feature flag.	P1	31st call returns heuristic-only.

5.11 Administration (ADM)

ID	Requirement	Pri	Acceptance criteria
FR-ADM-01	Store review queue with documents, map pin, and approve/reject with reason (emailed).	P0	Every action written to audit_logs.
FR-ADM-02	User and store search; suspend/unsuspend with reason.	P0	Suspended user sessions revoked immediately.
FR-ADM-03	Dispute queue with SLA timer (resolve within 48 h).	P0	Overdue disputes highlighted.
FR-ADM-04	Payments dashboard: captures, refunds, failed refunds, webhook lag, reconciliation mismatches.	P1	Mismatch list empty in steady state.
FR-ADM-05	Feature flags (DB-backed, cached 30 s): ai_advisor, web_push, saved_alerts, new_signups, maintenance_mode.	P1	Flag change takes effect ≤ 30 s without deploy.
FR-ADM-06	Admin accounts require TOTP 2FA.	P1	Admin routes reject sessions without 2FA claim.

6. Non-Functional Requirements

6.1 Performance and capacity

ID	Requirement	Target
NFR-PERF-01	Read APIs (nearby, lot detail) server latency	p95 ≤ 120 ms, p99 ≤ 300 ms
NFR-PERF-02	POST /orders server latency (excl. Razorpay call)	p95 ≤ 250 ms at 500 req/s
NFR-PERF-03	Realtime event: DB commit → client receive	p95 ≤ 500 ms, p99 ≤ 1 s
NFR-PERF-04	Concurrent WebSocket connections	10 000 on 2 realtime instances (2 vCPU / 4 GB each), CPU ≤ 70%
NFR-PERF-05	Flash-sale burst	2 000 reservation attempts on one lot within 10 s with zero oversell and error rate (non-business) ≤ 0.1%
NFR-PERF-06	Web vitals on mid-range Android over 4G	LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1; Lighthouse Performance ≥ 90, PWA installable
NFR-PERF-07	JS shipped on buyer home route	≤ 180 KB gzip (map library lazy-loaded)

6.2 Reliability and data integrity

ID	Requirement
NFR-REL-01	Availability target 99.5% monthly for API and realtime.
NFR-REL-02	Zero oversell and zero lost payment are invariants , checked by a scheduled integrity job every 10 min that pages on violation.
NFR-REL-03	All domain events are published via the transactional outbox: no event is lost if a process crashes after commit; consumers are idempotent (at-least-once delivery).
NFR-REL-04	Every scheduled transition has a delayed job and a once-per-minute sweeper, so a lost job delays a transition by ≤ 60 s but never skips it.
NFR-REL-05	Graceful shutdown: on SIGTERM stop accepting, drain HTTP (≤ 20 s), tell sockets to reconnect, finish in-flight jobs, then exit.
NFR-REL-06	Backups: Postgres point-in-time recovery enabled, daily snapshot kept 14 days. RPO ≤ 15 min, RTO ≤ 2 h. Restore drill performed once before launch and documented.
NFR-REL-07	External calls (Razorpay, email, LLM, push) have timeouts, retries with jitter, and circuit breakers; failure of email/LLM/push MUST NOT fail an order.

6.3 Scalability

- All processes are stateless and horizontally scalable; configuration only from environment variables (12-factor).
- Realtime fan-out through the Socket.io Redis adapter; rooms are scoped (geo cell, lot, user, store) so an event reaches only interested sockets.
- Read traffic for discovery MAY be routed to a read replica; anything inside a money flow MUST read from the primary.
- Hot-lot protection: per-lot `stock_changed` events are coalesced to at most 4 per second.
- Database connection pooling via PgBouncer (transaction mode) or the managed provider's pooler; each process has a bounded pool.

6.4 Security

ID	Requirement
NFR-SEC-01	OWASP ASVS Level 2 controls for auth, session, input validation and access control. OWASP Top 10 reviewed before launch (checklist in repo).
NFR-SEC-02	Every request body, query and socket payload validated with Zod schemas from packages/shared; unknown fields stripped.
NFR-SEC-03	Authorization enforced in the service layer with resource ownership checks (not only route guards). IDs are UUIDv7; no sequential IDs exposed.
NFR-SEC-04	CSRF: SameSite=Lax cookies + double-submit CSRF token on all state-changing requests. CORS allow-list of own origins only.
NFR-SEC-05	Security headers via Helmet (API) and the static-host header config (web): strict CSP (nonce-based), HSTS, X-Content-Type-Options, frame-ancestors none.
NFR-SEC-06	Rate limits (Redis sliding window): global 120 req/min/user, 60 req/min/IP for anonymous, auth endpoints per FR-AUTH-05, POST /orders 10/min/user, socket geo.subscribe 10/min/socket.
NFR-SEC-07	Secrets in the platform secret store; never in the repo. Gitleaks runs in CI. Dependencies scanned (npm audit + Dependabot); no critical vulnerabilities at release.
NFR-SEC-08	Card data never touches our servers (Razorpay Checkout). The system stores no card or UPI details.
NFR-SEC-09	PII fields (phone, email) are never logged; the logger redacts by key list. Audit log for all admin and money-changing actions, append-only.

6.5 Usability, accessibility and compatibility

- WCAG 2.1 AA: contrast $\geq 4.5:1$, full keyboard navigation, visible focus, screen-reader labels for live price (announced at most once per minute via `aria-live="polite"`).
- Respect `prefers-reduced-motion` (no price-tick animation) and `prefers-color-scheme` (light/dark themes).
- Supported: last 2 versions of Chrome, Safari (iOS 16.4+ for push), Firefox, Edge; Android Chrome 100+. Minimum width 320 px.
- All user-facing copy in English in v1, externalised for i18n (Hindi planned).

6.6 Observability

- Structured JSON logs (Pino) with `requestId`, `userId` (hashed), `traceId`; one log line per request and per job.
- OpenTelemetry tracing across HTTP $\rightarrow$ service $\rightarrow$ DB $\rightarrow$ queue $\rightarrow$ worker $\rightarrow$ socket emit.
- RED metrics per route and per job; business metrics: `reservations_total{result}`, `oversell_violations_total`, `payments_captured_total`, `webhook_lag_seconds`, `socket_connections`, `event_fanout_latency_seconds`.
- Error tracking with Sentry on web and server, release-tagged with source maps.

7. Edge-Case Catalogue (Strict)

Every row below MUST have a named automated test (unit, integration or e2e) before P1 sign-off. The test name MUST include the ID.

7.1 Concurrency and stock

ID	Scenario	Required behaviour
EC-01	Two buyers reserve the last unit in the same millisecond	Exactly one succeeds; other gets <code>INSUFFICIENT_STOCK</code> with current qty (0). Enforced by atomic guarded UPDATE.
EC-02	Buyer double-taps "Buy" (two requests, same Idempotency-Key)	One order; second request returns the identical stored response.
EC-03	Buyer opens two tabs and reserves in both (different keys)	Allowed only within BR-BUY-05/06 limits; per-(user, lot) advisory lock serialises the check.
EC-04	Hold expires at the same moment the payment webhook arrives	Both paths use compare-and-set on order status; exactly one wins. If expiry won, late-capture path runs (re-reserve or refund).
EC-05	Seller increases qty while buyers are reserving	Both are single-row atomic updates; <code>qty_available</code> and <code>qty_total</code> stay consistent (CHECK constraint).
EC-06	Seller cancels the lot while a buyer is at checkout	Lot → <code>CANCELLED</code> ; <code>PENDING</code> orders → <code>CANCELLED_BY_SELLER</code> ; any payment captured later is refunded automatically.
EC-07	Reservation arrives 1 ms after <code>ends_at</code>	Rejected <code>LOT_NOT_LIVE</code> (DB clock decides), even if scheduler has not yet flipped status.

7.2 Time and clocks

ID	Scenario	Required behaviour
EC-08	Client clock is 3 minutes fast	UI uses server offset from <code>time.sync</code> ; if price at server is higher than <code>expectedUnitPrice</code> → <code>PRICE_MISMATCH</code> with fresh price; UI refreshes and asks to confirm.
EC-09	Request lands exactly on a price-drop boundary	Server price is authoritative; buyer charged the lower-or-equal price; never more than shown (BR-BUY-03).
EC-10	Worker is down when <code>starts_at</code> passes	Sweeper promotes the lot within 60 s of recovery; price formula still uses original <code>starts_at</code> (no extra time at high price).
EC-11	Tab sleeps in background for 20 min then wakes	On <code>visibilitychange</code> the client re-syncs time and refetches lot snapshots before showing prices.
EC-12	Lot scheduled across midnight / DST	All logic in UTC; IST has no DST; display formatting tested for 23:30 → 00:30 lots.

7.3 Payments

ID	Scenario	Required behaviour
EC-13	Webhook delivered 3 times	Unique <code>provider_event_id</code> → processed once; later deliveries return 200.
EC-14	Webhook never arrives	Reconciliation job (FR-PAY-04) detects capture and confirms/refunds within 10 min.
EC-15	Webhook arrives before the API has stored the Razorpay order id (race)	Event stored; processor retries with backoff until order found (max 10 min) then alerts.
EC-16	Payment amount in webhook ≠ order total	Do not confirm; mark order <code>PAYMENT_MISMATCH</code> flag, auto-refund, alert admin.
EC-17	Buyer pays twice for one order (two payment attempts both captured)	First capture confirms; every additional capture on the same Razorpay order is refunded automatically.

ID	Scenario	Required behaviour
EC-18	Refund API returns 5xx / timeout	Refund row stays PENDING; retried with backoff; idempotent by our refund id; after 8 failures → admin alert, never silently dropped.
EC-19	payment.failed then later payment.captured on same order (buyer retried)	Failed attempt recorded; capture confirms order if hold still valid, else late-capture path.
EC-20	Signature invalid or body tampered	400, event not stored as trusted, security alert metric incremented.

7.4 Realtime and network

ID	Scenario	Required behaviour
EC-21	Socket disconnects for 30 s and events are missed	On reconnect client refetches REST snapshot (with version) then resumes applying events; stale events (lower version) ignored.
EC-22	Events arrive out of order	Each lot event carries monotonically increasing version; client applies only if version > local.
EC-23	Realtime instance killed during load	Clients reconnect to another instance with exponential backoff (0.5 s → 10 s, jitter) and full-jitter; p95 reconnect ≤ 5 s.
EC-24	Access token expires while socket is connected	Server emits auth.expired; client refreshes via HTTP and re-authenticates the socket without dropping rooms.
EC-25	User moves 4 km while browsing	Client recomputes geohash cells when moved > 500 m; leaves/joins diff of rooms; list refetched.
EC-26	Buyer on flaky 3G at checkout	Reservation POST retried with the same Idempotency-Key; UI shows "confirming..." not a second charge.

7.5 Pickup, abuse and data

ID	Scenario	Required behaviour
EC-27	Staff enters a valid code from a different store	Rejected ORDER_NOT_FOUND (scoped lookup; do not reveal existence).
EC-28	Buyer shows screenshot of someone else's QR	Code is single-use; first verification wins; second returns alreadyVerified with time.
EC-29	Buyer reserves and abandons repeatedly to block stock	BR-BUY-06 and BR-BUY-07 limits; metric of hold-abandon rate per user.
EC-30	Seller creates fake buyer account to inflate sales	Self-purchase blocked by store membership; same-device/same-payment-contact heuristics flagged to admin (P2).
EC-31	Image upload with a script disguised as JPEG	Magic-byte check + re-encode by Sharp; original discarded.
EC-32	Store is suspended while it has CONFIRMED orders	Orders stay valid and pickable; new reservations blocked; buyers notified.
EC-33	Account deleted with open orders	Deletion blocked until orders are terminal (OPEN_ORDERS_EXIST).
EC-34	Lot title contains emoji, Hindi, 80-char limit	Length counted in grapheme clusters; stored as UTF-8; rendered correctly.

8. Technology Stack

Versions: use the latest stable release of each tool at project start and pin exact versions in the lockfile. Upgrades happen deliberately via Renovate/Dependabot PRs, never implicitly.

8.1 Frontend

Concern	Choice	Why	Rejected
Framework	React + Vite, TypeScript strict	Engineer already knows React; Vite gives instant dev server and small builds; static SPA is cheap to host on a CDN	Next.js (new framework to learn; SSR not essential)
Routing	React Router	Standard client routing with lazy-loaded route groups for buyer, seller and admin	TanStack Router (less familiar)
Link previews	Server share route (/s/lot/:id) that renders Open Graph tags then redirects to the SPA	Keeps WhatsApp/social previews without server-side rendering the app	Prerendering service
Styling / UI kit	Tailwind CSS + shadcn/ui (Radix primitives) + lucide-react	Accessible primitives you own the code of; consistent design tokens	MUI (heavy, generic look)
Motion	Motion (Framer Motion)	Price-tick, list reorder and sheet animations; honours reduced-motion	CSS-only (hard for layout animation)
Server state	TanStack Query	Caching, retries, socket-driven cache updates via setQueryData	Redux Toolkit Query
Client state	Zustand	Tiny store for location, time offset, UI state	Context sprawl
Forms	React Hook Form + Zod (shared schemas)	Same validation as server; per-field errors	Formik
Maps	MapLibre GL JS + MapTiler/OSM tiles + supercluster	Open source, no per-load Google billing, vector tiles, clustering	Google Maps JS (cost, key exposure)
Charts	Recharts	Price curve step chart, analytics	Chart.js
PWA	Servist (service worker) + Web App Manifest	Installable, offline shell, push handling	Workbox by hand
Realtime client	socket.io-client (websocket transport only)	Matches server; auto-reconnect with backoff	Raw WebSocket

8.2 Backend

Concern	Choice	Why	Rejected
Runtime	Node.js LTS	One language end to end; strong async I/O for sockets	—
Framework	Express + TypeScript	Engineer already knows it; minimal and universal. Structure is enforced by our own module rules, lint rules and review agents (a good interview talking point)	NestJS (new framework to learn)
Database	PostgreSQL 16 + PostGIS	ACID money flows, row-level atomic updates, geo queries with GiST index	MongoDB (weaker multi-doc transactions, no PostGIS)
ORM / migrations	Drizzle ORM + drizzle-kit	SQL-first, typed, first-class raw SQL for guarded updates and PostGIS, no query engine binary	Prisma (weak PostGIS support, raw-SQL-heavy for locking)
Cache / pub-sub / locks	Redis 7	Socket.io adapter, rate limits, idempotency locks, hot caches	Memcached (no pub/sub)

Concern	Choice	Why	Rejected
Queues / scheduling	BullMQ	Delayed jobs for lot start/end, retries with backoff, job ids for idempotency	Cron-only (no per-lot precision)
Realtime	Socket.io + @socket.io/redis-adapter	Rooms, acks, reconnection, horizontal fan-out	Pusher/Ably (hides the engineering)
Payments	Razorpay Orders API, Checkout, Webhooks, Refunds (Route in P2)	Indian UPI/cards/netbanking, test mode, webhooks	Stripe (limited Indian UPI support for this use)
Validation	Zod (in packages/shared)	Single schema for API, sockets, forms and LLM output	class-validator (duplicated types)
Auth	argon2, jose (JWT), otplib (TOTP), openid-client (Google)	Own the auth to show you understand it; standards-based	Clerk/Auth0 (hides the engineering)
Email	Resend (or AWS SES) + React Email templates	Simple API, good deliverability, typed templates	SMTP by hand
Push	web-push (VAPID)	Standard Web Push, no vendor lock-in	Firebase Cloud Messaging
Object storage	Cloudflare R2 (S3 API) + Sharp in worker	Zero egress fees, presigned uploads, CDN	Storing images in DB
AI	LLM provider behind a LlmClient interface (default: a small, fast Claude model)	Swappable, testable with a fake; strict JSON validation	Calling SDK directly from controllers

8.3 Quality, delivery and operations

Concern	Choice
Monorepo	pnpm workspaces + Turborepo (cached builds and tests)
Code quality	ESLint (typescript-eslint strict), Prettier, Husky + lint-staged, Commitlint (Conventional Commits)
Unit / property tests	Vitest + fast-check
Integration tests	Vitest + Supertest + Testcontainers (real Postgres/PostGIS and Redis in Docker)
E2E tests	Playwright (mobile + desktop viewports), Razorpay test mode
Load tests	k6 (HTTP scenarios) + Artillery with socket.io engine (connection storms)
Observability	OpenTelemetry SDK → Grafana Cloud (Prometheus, Loki, Tempo) · Pino logs · Sentry
Containers / IaC	Docker (multi-stage, distroless runtime), docker-compose for local, Terraform for AWS tier
CI/CD	GitHub Actions: lint → typecheck → unit → integration → build → e2e → image → deploy staging → smoke → manual promote
Security scanning	Gitleaks, npm audit, Dependabot, Trivy image scan, OWASP ZAP baseline on staging
AI-assisted development	Claude Code with root and nested CLAUDE.md, MCP servers (Context7, Playwright, GitHub, local Postgres read-only, Google Stitch), 13 skills, 6 review/test agents and guard-rail hooks — see the companion <i>*AI Engineering Workflow*</i> document

9. High-Level Design (HLD)

9.1 Architecture style

SurplusDrop is a **modular monolith**: one Express + TypeScript codebase with strict module boundaries, deployed as three **process roles** with separate entry points — `api.ts`, `realtime.ts` and `worker.ts`. Each role scales independently. Modules communicate only through their public service interface or through domain events on the outbox; a module **MUST NOT** query another module's tables. This gives the operational simplicity of one deployable with a clean path to extract services later (see ADR-001).

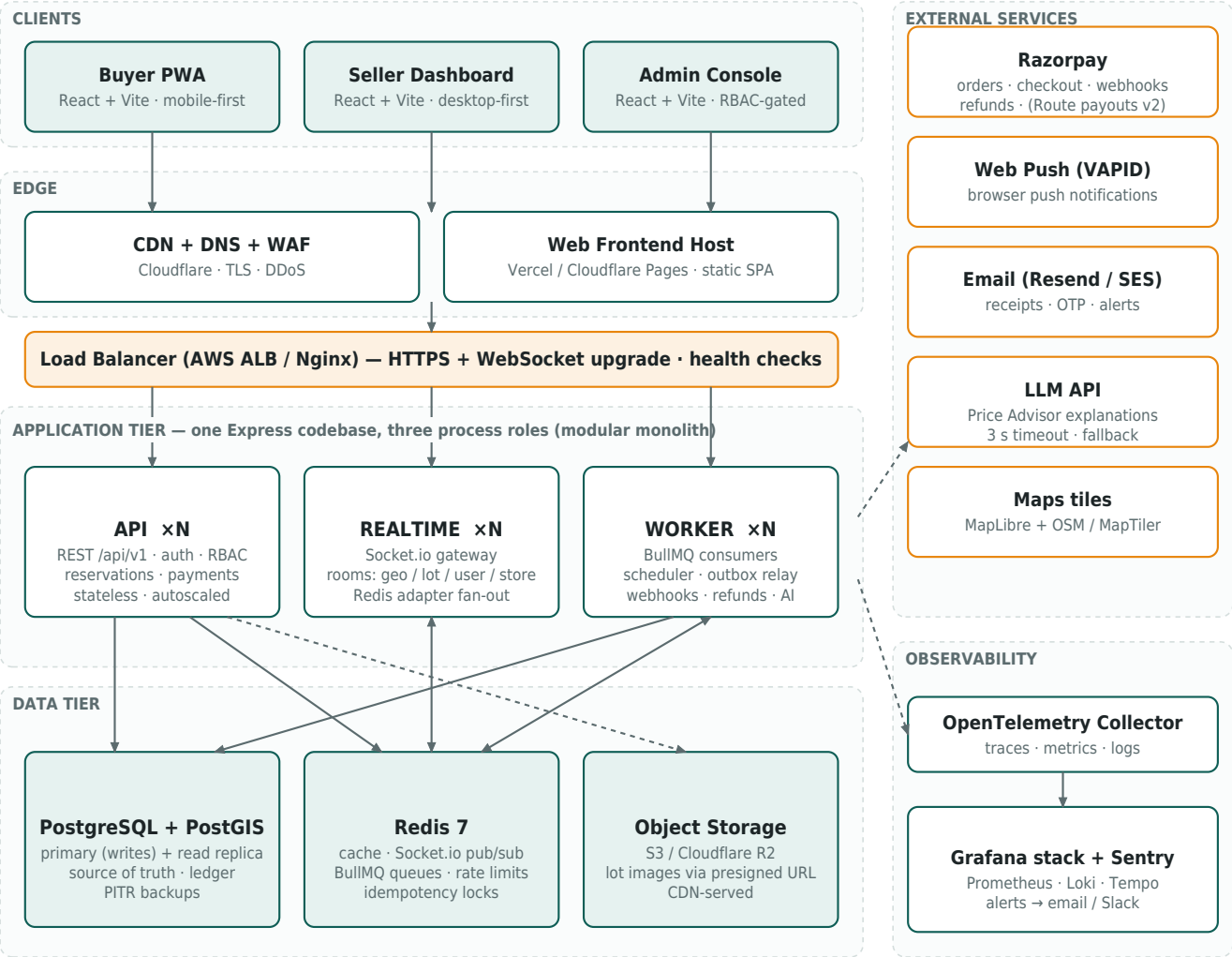


Figure 1 — System context and containers.

9.2 Components

Component	Responsibility	Scales on
Web (React + Vite, static on CDN)	Buyer PWA, seller dashboard, admin console; service worker for push and offline shell	Edge / CDN automatically
Share route (API role)	Open Graph HTML for shared lot links, then redirect into the SPA	With API
API role	REST endpoints, auth, validation, business rules, transactions, outbox writes, Razorpay order creation, webhook intake	CPU / request rate

Component	Responsibility	Scales on
Realtime role	Socket authentication, room membership, time sync, receiving published domain events from Redis and emitting to rooms	Connection count
Worker role	Outbox relay, lot scheduler + sweeper, payment event processing, refunds, reconciliation, notifications, image processing, AI calls, analytics rollups	Queue depth / lag
PostgreSQL + PostGIS	System of record: users, stores, lots, orders, payments, ledger, outbox, audit	Vertical + read replica
Redis	Socket.io adapter pub/sub, BullMQ, rate limits, idempotency locks, short-lived caches	Vertical (cluster in P2)
Object storage + CDN	Lot and store images, dispute evidence	Managed

9.3 Module map

Module	Owns tables	Publishes events
identity	users, sessions, refresh_tokens, email_otps, password_resets	user.registered, user.suspended
stores	stores, store_members, store_hours, store_strikes	store.approved, store.suspended
lots	lots, lot_images	lot.published, lot.started, lot.stock_changed, lot.sold_out, lot.ended, lot.cancelled
orders	orders	order.reserved, order.confirmed, order.expired, order.cancelled, order.picked_up, order.no_show
payments	payments, payment_events, refunds	payment.captured, refund.succeeded, refund.failed
ledger	ledger_accounts, ledger_transactions, ledger_entries, payouts	—
notifications	notifications, push_subscriptions, follows, saved_alerts	—
trust	ratings, disputes, reports	dispute.resolved
analytics	lot_results, store_daily_stats	—
ai	ai_suggestions	—
platform	outbox_events, idempotency_keys, audit_logs, feature_flags	—

9.4 Critical flow — reserve and pay

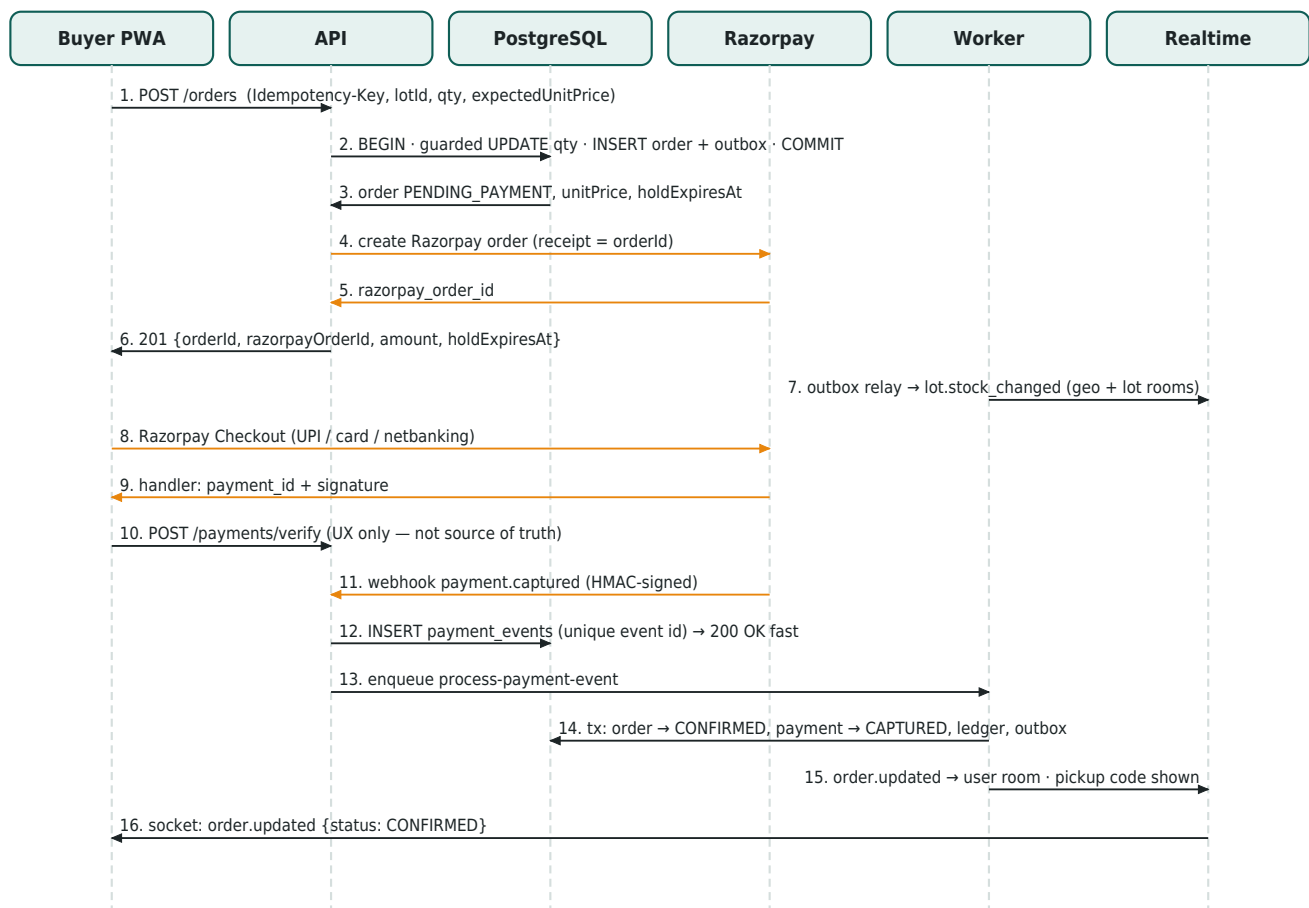


Figure 2 — Reservation and payment. The webhook (steps 11-14), not the browser callback, confirms the order.

9.5 Realtime design

Rooms

Room	Joined by	Receives
geo:{geohash5}	Any client, for its cell + 8 neighbours ($\approx 5 \text{ km} \times 5 \text{ km}$ cells, max 9 rooms)	lot.published, lot.started, lot.stock_changed, lot.ended (summary payload)
lot:{lotId}	Clients on a lot detail or checkout screen	Full lot events incl. watcher count
user:{userId}	Authenticated sockets, automatically	order.updated, notification.created, auth.expired
store:{storeId}	Store owner/staff sockets, after membership check	order.confirmed (new order), order.picked_up, lot state for dashboard

Why prices are not broadcast

The price changes on a fixed schedule, so broadcasting every tick would multiply traffic by the number of lots and watchers for zero information. Instead each lot snapshot carries its price configuration and serverTime; every client computes the price locally with the shared priceAt function. The server only broadcasts **state changes** (stock, status). This cuts realtime traffic by roughly two orders of magnitude and is a central design decision (ADR-004).

Time synchronisation

On connect and every 5 minutes the client sends time.sync {t0}; the server acks {t0, serverTime}; the client records t3 and computes $\text{offset} = \text{serverTime} - (t0 + t3) / 2$, keeping the median of the last 5 samples with the lowest round-trip. All countdowns use Date.now() + offset.

Consistency on the client

- Every lot row has an integer `version` incremented on every change; every lot event carries it. Clients apply an event only if `event.version > cached.version`.
- On (re)connect the client first re-joins rooms, then refetches the REST snapshot, then applies buffered events newer than the snapshot. No gaps, no stale overwrites.
- Events update TanStack Query caches directly (`setQueryData`), so lists and detail pages stay in sync without refetch storms.

Delivery path

Domain write + `outbox_events` insert commit in one transaction → worker relay claims rows with `FOR UPDATE SKIP LOCKED` (batch 100, every 100 ms or on `LISTEN/NOTIFY`) → publishes to Redis channel `events` → every realtime instance receives it, maps event type to rooms, emits via the Redis adapter → marks outbox row `published_at`. Delivery is at-least-once; client-side version checks make duplicates harmless.

9.6 Consistency and failure strategy

Mechanism	Where used	Guarantees
Guarded atomic UPDATE (compare-and-set)	Stock decrement, every state transition	No oversell, no double transition without explicit locks
Idempotency keys (DB unique + stored response)	POST /orders, cancel, verify, refunds	Client retries never duplicate side effects
Transactional outbox	All domain events	No lost events after commit; no events for rolled-back work
Unique provider event id	Razorpay webhooks	Exactly-once processing on at-least-once delivery
Delayed job + sweeper	Lot start/end, hold expiry, no-show	Timely transitions that self-heal after outages
Reconciliation job	Payments	Recovers from lost webhooks and partial failures
Double-entry ledger	All money movement	Every paisa traceable; balances provable
Integrity checker	Every 10 min	Alerts on any invariant violation (oversell, orphan capture, unbalanced ledger)

9.7 Caching strategy

Data	Cache	TTL / invalidation
Nearby-lots query per (geohash6, radius, filters)	Redis	3 s TTL; stock changes arrive via sockets, so short staleness is invisible
Lot detail snapshot	Redis, keyed by id + version	Invalidated on version change
Share-preview HTML for a lot	Redis	60 s TTL; invalidated on lot version change
Store profile + rating	Redis	5 min; invalidated on update or new rating
Feature flags	In-process memory	30 s
Anything in a money flow	Never cached	Always read from the primary database

9.8 Scaling plan and known bottlenecks

Bottleneck	Symptom	Mitigation
Hot lot (thousands reserving one row)	Row lock contention, p99 rises	Single short UPDATE (no read-then-write); keep transaction < 5 ms; P2: Redis pre-decrement token bucket in front of DB
Fan-out on a busy geo cell	Event latency rises with watchers	Coalesce stock events ($\leq 4/s$ per lot); summary payloads in geo rooms; full payload only in lot rooms
Connection storm after deploy	Realtime CPU spike	Rolling deploy, randomised reconnect jitter, connection rate limit per IP
DB connections	Pool exhaustion	Pooler in transaction mode, bounded pools per process, statement timeout 5 s
Nearby query at scale	Slow geo scans	GiST index on <code>stores.location</code> , partial index on live lots, read replica
Webhook bursts	Slow ack → Razorpay retries	Store-and-ack in < 200 ms, process in worker

9.9 Deployment topology

Tier	Web	API / Realtime / Worker	Postgres	Redis	Purpose
A — Launch (low cost)	Vercel	Fly.io or Railway: 2 × api, 2 × realtime, 1 × worker containers	Managed Postgres with PostGIS (Neon or Supabase)	Managed Redis (Railway / Upstash with TCP)	Public demo, real users, $\leq ₹3$ 000/month
B — Showcase (AWS)	Vercel or S3 + CloudFront	ECS Fargate services behind ALB, autoscaling on CPU and custom metrics	RDS PostgreSQL + read replica, PITR	ElastiCache Redis	Load-test runs and resume evidence; spun up with Terraform, torn down after

Environments: **local** (docker-compose: Postgres+PostGIS, Redis, MinIO, Mailpit), **staging** (tier A, Razorpay test mode, seeded data) and **production** (tier A, Razorpay live mode after KYC; test mode otherwise, clearly labelled).

9.10 Architecture Decision Records (summary)

ADR	Decision	Main reason
ADR-001	Modular monolith with 3 process roles, not microservices	One engineer; boundaries give the benefits without distributed-transaction cost
ADR-002	PostgreSQL is the only source of truth for stock (no Redis stock counter in v1)	Atomic guarded UPDATE is correct and fast enough to 500 req/s on one lot
ADR-003	Webhook is the source of truth for payment	Browser callbacks are lost when users close tabs
ADR-004	Clients compute price locally; server broadcasts only state changes	Orders-of-magnitude less realtime traffic; deterministic shared function
ADR-005	Transactional outbox for all events	Prevents lost or phantom events
ADR-006	PWA instead of native apps	Recruiters open links; push/geolocation/installability cover the needs
ADR-007	Socket.io websocket-only transport	No sticky sessions needed at the load balancer
ADR-008	LLM explains, heuristic decides prices	Determinism, testability and safety for money-affecting numbers
ADR-009	Drizzle over Prisma	PostGIS and raw SQL for atomic updates are first-class
ADR-010	React + Vite and Express instead of Next.js and NestJS	Build with known tools; spend learning time on real-time, payments and scale
ADR-011	pnpm + Turborepo monorepo with <code>.claude/</code> at the root	Shared contracts; one place for AI tooling, hooks and review agents

ADR	Decision	Main reason
ADR-012	Delivery-ready order model (fulfilment_type, extra_charges)	Delivery can be added later without rewriting orders or payments

10. Low-Level Design (LLD)

10.1 Repository structure

Monorepo layout

surplusdrop/	
├─ CLAUDE.md	root instructions for Claude Code (imports project-context.md)
├─ project-context.md	living project memory for new sessions / models
├─ DESIGN.md	design tokens (extracted from Google Stitch)
├─ .mcp.json	MCP servers (uses \${ENV_VAR} placeholders; no secrets)
├─ .claude/	
│ ├─ settings.json	permissions + hooks
│ ├─ hooks/	protect-files · block-dangerous · format · stop-check (.mjs)
│ ├─ skills/<name>/SKILL.md	/new-feature /new-endpoint /local-check /feature-test /review ...
│ └─ agents/*.md	code · security · money reviewers, test-writer, qa-tester, ui-reviewer
├─ apps/	
│ └─ web/	React + Vite + React Router (+ CLAUDE.md)
│ ├─ src/app/	router, providers, layouts
│ ├─ src/pages/	buyer/ seller/ admin/ (lazy-loaded route groups)
│ ├─ src/features/	lots, checkout, orders, map, seller, admin (components+hooks+api)
│ ├─ src/components/ui/	shadcn/ui primitives
│ ├─ src/lib/	api client, socket, time-sync, query keys, errors
│ ├─ src/sw/	service worker (Serwist)
│ └─ e2e/	Playwright specs
│ └─ server/	Express + TypeScript (+ CLAUDE.md)
│ ├─ src/entry/	api.ts · realtime.ts · worker.ts
│ ├─ src/modules/<module>/	routes · controller · service · repository · schemas · domain/
│ ├─ src/platform/	config, db, redis, outbox, idempotency, auth, errors, logger
│ ├─ src/realtime/	socket server, rooms, event-room router, time sync
│ ├─ src/jobs/	BullMQ processors + sweepers
│ ├─ src/share/	Open Graph share route for lot links
│ └─ test/	integration/ (Testcontainers), fixtures/razorpay/
├─ packages/	
│ └─ shared/	zod schemas, DTO types, pricing.ts, error codes, event contracts
│ └─ db/	drizzle schema, migrations, seed
│ └─ ui-tokens/	design tokens consumed by Tailwind
│ └─ config/	eslint, tsconfig, prettier presets
├─ infra/	docker-compose.yml, Dockerfiles, terraform/, k6/, grafana/
├─ docs/	SRS, AI workflow guide, adr/, runbooks/, postmortems/, load-testing.md
├─ .github/workflows/	ci.yml, deploy.yml, load-test.yml
└─ package.json · pnpm-workspace.yaml · turbo.json · .env.example	

10.2 Layering rules inside a module

Layer	Contains	May depend on	MUST NOT
Routes + Controller / Socket handler	Express routes and middleware, Zod parsing, auth checks	Application service	Contain business rules or SQL
Application service	Use cases (reserveLot, confirmPayment), transactions, outbox writes	Domain, repositories, other modules' public services	Import another module's repository
Domain	Pure functions and types: price, state transitions, policy checks	packages/shared	Do I/O, read clock directly (clock is injected)
Repository	Drizzle queries and raw SQL for this module's tables	DB client	Be called from controllers

10.3 Database schema (core DDL)

Supporting tables not shown in full: email_otps, password_resets, store_hours, store_strikes, lot_images, notifications, push_subscriptions, follows, saved_alerts, ratings, disputes, reports, payouts, lot_results, store_daily_stats, ai_suggestions, feature_flags. Each follows the same conventions: UUIDv7 primary key, created_at/updated_at, CHECK constraints mirroring business rules, and indexes for every foreign key used in a WHERE clause.

Identity and stores

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS citext;

CREATE TYPE user_status AS ENUM ('ACTIVE', 'SUSPENDED', 'DELETED');
CREATE TYPE store_status AS ENUM ('PENDING_REVIEW', 'APPROVED', 'REJECTED', 'SUSPENDED');
CREATE TYPE lot_status AS ENUM ('DRAFT', 'SCHEDULED', 'LIVE', 'SOLD_OUT', 'ENDED', 'CANCELLED');
CREATE TYPE order_status AS ENUM ('PENDING_PAYMENT', 'CONFIRMED', 'PICKED_UP', 'NO_SHOW',
                                   'EXPIRED', 'CANCELLED_BY_BUYER', 'CANCELLED_BY_SELLER');
CREATE TYPE payment_status AS ENUM ('CREATED', 'CAPTURED', 'FAILED', 'REFUND_PENDING',
                                     'PARTIALLY_REFUNDED', 'REFUNDED');
CREATE TYPE refund_status AS ENUM ('PENDING', 'PROCESSING', 'SUCCEEDED', 'FAILED');

CREATE TABLE users (
  id          uuid PRIMARY KEY,          -- UUIDv7 generated in app
  email       citext NOT NULL UNIQUE,
  email_verified_at timestampz,
  password_hash text,                    -- null for OAuth-only
  name        text NOT NULL CHECK (char_length(name) BETWEEN 2 AND 60),
  is_admin    boolean NOT NULL DEFAULT false,
  totp_secret_enc bytea,                 -- admins only, encrypted
  status      user_status NOT NULL DEFAULT 'ACTIVE',
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  deleted_at  timestampz
);

CREATE TABLE refresh_tokens (
  id          uuid PRIMARY KEY,
  user_id     uuid NOT NULL REFERENCES users(id),
  family_id   uuid NOT NULL,
  token_hash  bytea NOT NULL UNIQUE,     -- SHA-256 of token
  user_agent  text, ip_hash bytea,
  expires_at  timestampz NOT NULL,
  rotated_at  timestampz,                -- set when replaced
  revoked_at  timestampz,
  created_at  timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ON refresh_tokens (user_id) WHERE revoked_at IS NULL;

CREATE TABLE stores (
  id          uuid PRIMARY KEY,
  name        text NOT NULL CHECK (char_length(name) BETWEEN 2 AND 60),
  slug        text NOT NULL UNIQUE,
  category    text NOT NULL,
  description  text CHECK (char_length(description) <= 500),
  phone       text NOT NULL,
  address     text NOT NULL,
```

```

location          geography(Point, 4326) NOT NULL,
fssai_number      char(14) CHECK (fssai_number ~ '^[0-9]{14}$'),
gstn              char(15) CHECK (gstn ~ '^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]$'),
status            store_status NOT NULL DEFAULT 'PENDING_REVIEW',
status_reason      text,
commission_bps     int NOT NULL DEFAULT 1000 CHECK (commission_bps BETWEEN 0 AND 3000),
rating_sum         int NOT NULL DEFAULT 0, rating_count int NOT NULL DEFAULT 0,
holiday_mode       boolean NOT NULL DEFAULT false,
created_at         timestampz NOT NULL DEFAULT now(),
updated_at         timestampz NOT NULL DEFAULT now()
);
CREATE INDEX stores_location_gix ON stores USING gist (location);

CREATE TABLE store_members (
  store_id uuid NOT NULL REFERENCES stores(id),
  user_id  uuid NOT NULL REFERENCES users(id),
  role     text NOT NULL CHECK (role IN ('OWNER','STAFF')),
  PRIMARY KEY (store_id, user_id)
);

```

Lots and orders

```

CREATE TABLE lots (
  id                uuid PRIMARY KEY,
  store_id          uuid NOT NULL REFERENCES stores(id),
  title             text NOT NULL CHECK (char_length(title) BETWEEN 5 AND 80),
  description        text CHECK (char_length(description) <= 500),
  category          text NOT NULL,
  dietary_tags       text[] NOT NULL DEFAULT '{}',
  mrp               int,
  start_price        int NOT NULL CHECK (start_price BETWEEN 1000 AND 1000000),
  floor_price        int NOT NULL CHECK (floor_price >= 100),
  decrement_amount   int NOT NULL CHECK (decrement_amount >= 100),
  decrement_interval_sec int NOT NULL CHECK (decrement_interval_sec IN (60,120,300,600,900)),
  qty_total          int NOT NULL CHECK (qty_total BETWEEN 1 AND 500),
  qty_available      int NOT NULL,
  max_qty_per_order  int NOT NULL DEFAULT 5 CHECK (max_qty_per_order BETWEEN 1 AND 10),
  starts_at          timestampz NOT NULL,
  ends_at            timestampz NOT NULL,
  pickup_start       timestampz NOT NULL,
  pickup_end         timestampz NOT NULL,
  best_before        timestampz,
  status             lot_status NOT NULL DEFAULT 'DRAFT',
  end_reason          text,          -- TIME | SOLD_OUT | EARLY | CANCELLED | MODERATION
  version            int NOT NULL DEFAULT 1,
  created_at          timestampz NOT NULL DEFAULT now(),
  updated_at          timestampz NOT NULL DEFAULT now(),
  CONSTRAINT qty_bounds CHECK (qty_available BETWEEN 0 AND qty_total),
  CONSTRAINT mrp_ok    CHECK (mrp IS NULL OR mrp >= start_price),
  CONSTRAINT floor_lt_start CHECK (floor_price * 10 <= start_price * 9),
  CONSTRAINT dec_range CHECK (decrement_amount <= start_price - floor_price),
  CONSTRAINT duration  CHECK (ends_at - starts_at BETWEEN interval '15 min' AND interval '12 hours'),
  CONSTRAINT pickup_window CHECK (pickup_start >= starts_at
                                AND pickup_end >= ends_at + interval '30 min'
                                AND pickup_end <= ends_at + interval '12 hours'
                                AND pickup_end - pickup_start >= interval '30 min'),
  CONSTRAINT per_order CHECK (max_qty_per_order <= qty_total)
);
CREATE INDEX lots_active_idx ON lots (store_id, ends_at) WHERE status IN ('SCHEDULED','LIVE','SOLD_OUT');
CREATE INDEX lots_status_time_idx ON lots (status, starts_at, ends_at);
CREATE INDEX lots_title_trgm ON lots USING gin (title gin_trgm_ops);

CREATE TABLE orders (
  id                uuid PRIMARY KEY,
  lot_id            uuid NOT NULL REFERENCES lots(id),
  store_id          uuid NOT NULL REFERENCES stores(id),          -- denormalised for scoping
  buyer_id         uuid NOT NULL REFERENCES users(id),
  qty               int NOT NULL CHECK (qty BETWEEN 1 AND 10),
  unit_price         int NOT NULL CHECK (unit_price >= 100),
  fulfilment_type    text NOT NULL DEFAULT 'PICKUP' CHECK (fulfilment_type IN ('PICKUP')), -- ADR-012

```

```

extra_charges    int NOT NULL DEFAULT 0 CHECK (extra_charges = 0), -- future delivery fee
total_amount     int NOT NULL,
status           order_status NOT NULL DEFAULT 'PENDING_PAYMENT',
hold_expires_at  timestampz NOT NULL,
pickup_code_hash bytea, -- HMAC of 6-digit code
pickup_code_enc  bytea, -- encrypted, shown to buyer
confirmed_at     timestampz, picked_up_at timestampz, cancelled_at timestampz,
verified_by      uuid REFERENCES users(id),
flags            text[] NOT NULL DEFAULT '{}', -- e.g. PAYMENT_MISMATCH
version          int NOT NULL DEFAULT 1,
created_at       timestampz NOT NULL DEFAULT now(),
updated_at       timestampz NOT NULL DEFAULT now(),
CONSTRAINT total_ok CHECK (total_amount = qty * unit_price + extra_charges)
);
CREATE INDEX orders_buyer_idx ON orders (buyer_id, created_at DESC);
CREATE INDEX orders_lot_idx ON orders (lot_id, status);
CREATE INDEX orders_pending_idx ON orders (hold_expires_at) WHERE status = 'PENDING_PAYMENT';
CREATE INDEX orders_confirmed_idx ON orders (store_id, status) WHERE status = 'CONFIRMED';
CREATE UNIQUE INDEX orders_store_code_uq ON orders (store_id, pickup_code_hash)
WHERE status = 'CONFIRMED';

```

Payments, ledger and platform

```

CREATE TABLE payments (
  id          uuid PRIMARY KEY,
  order_id    uuid NOT NULL REFERENCES orders(id),
  provider    text NOT NULL DEFAULT 'razorpay',
  provider_order_id text NOT NULL UNIQUE,
  provider_payment_id text UNIQUE,
  amount      int NOT NULL,
  amount_refunded int NOT NULL DEFAULT 0 CHECK (amount_refunded BETWEEN 0 AND amount),
  fee int, tax int, method text,
  status      payment_status NOT NULL DEFAULT 'CREATED',
  captured_at timestampz,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now()
);
CREATE INDEX payments_order_idx ON payments (order_id);

CREATE TABLE payment_events ( -- raw webhook inbox
  id          uuid PRIMARY KEY,
  provider_event_id text NOT NULL UNIQUE, -- x-razorpay-event-id
  event_type  text NOT NULL,
  payload     jsonb NOT NULL,
  received_at timestampz NOT NULL DEFAULT now(),
  processed_at timestampz,
  attempts    int NOT NULL DEFAULT 0,
  last_error  text
);
CREATE INDEX payment_events_unprocessed ON payment_events (received_at) WHERE processed_at IS NULL;

CREATE TABLE refunds (
  id          uuid PRIMARY KEY,
  payment_id  uuid NOT NULL REFERENCES payments(id),
  order_id    uuid NOT NULL REFERENCES orders(id),
  amount      int NOT NULL CHECK (amount > 0),
  reason      text NOT NULL, -- SELLER_CANCELLED | LATE_CAPTURE_NO_STOCK | DUPLICATE_CAPTURE
                                -- | AMOUNT_MISMATCH | DISPUTE_FULL | DISPUTE_PARTIAL
  provider_refund_id text UNIQUE,
  status      refund_status NOT NULL DEFAULT 'PENDING',
  attempts    int NOT NULL DEFAULT 0,
  next_attempt_at timestampz,
  created_at  timestampz NOT NULL DEFAULT now(),
  UNIQUE (order_id, reason, payment_id) -- one refund per cause per payment
);

-- Double-entry ledger -----
CREATE TABLE ledger_accounts (
  id      uuid PRIMARY KEY,
  code    text NOT NULL UNIQUE -- 'gateway_clearing', 'platform_revenue', 'gateway_fees',
                                -- 'refunds_payable', 'seller_payable:{storeId}'

```

```

);
CREATE TABLE ledger_transactions (
  id uuid PRIMARY KEY, kind text NOT NULL, order_id uuid REFERENCES orders(id),
  idempotency_key text NOT NULL UNIQUE, created_at timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE ledger_entries (
  id bigserial PRIMARY KEY,
  transaction_id uuid NOT NULL REFERENCES ledger_transactions(id),
  account_id uuid NOT NULL REFERENCES ledger_accounts(id),
  amount bigint NOT NULL CHECK (amount <> 0) -- +debit / -credit, pause
);
-- Deferred constraint trigger: SUM(amount) per transaction_id MUST equal 0 at COMMIT.

-- Platform -----
CREATE TABLE outbox_events (
  id bigserial PRIMARY KEY,
  aggregate text NOT NULL, aggregate_id uuid NOT NULL,
  type text NOT NULL, version int, payload jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  published_at timestamptz
);
CREATE INDEX outbox_unpublished ON outbox_events (id) WHERE published_at IS NULL;

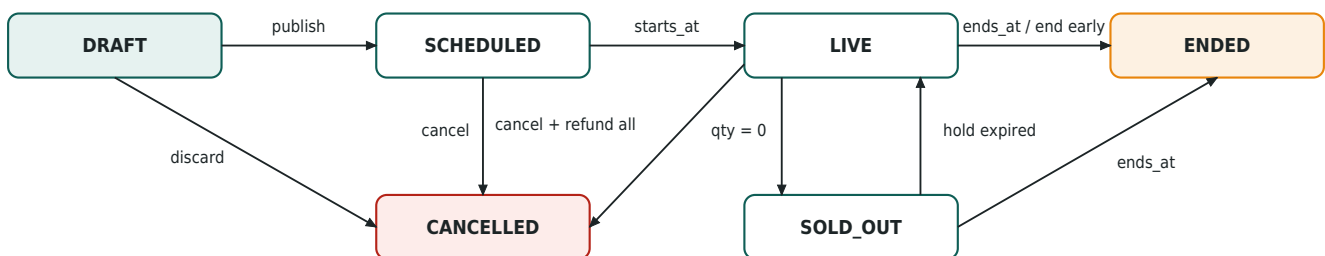
CREATE TABLE idempotency_keys (
  key text NOT NULL, user_id uuid NOT NULL, route text NOT NULL,
  request_hash bytea NOT NULL, status_code int, response jsonb,
  locked_until timestamptz, created_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, route, key)
);

CREATE TABLE audit_logs (
  id bigserial PRIMARY KEY, actor_id uuid, action text NOT NULL,
  target_type text NOT NULL, target_id uuid, data jsonb,
  ip_hash bytea, created_at timestamptz NOT NULL DEFAULT now()
); -- app role has INSERT + SELECT only (no UPDATE/DELETE grants)

```

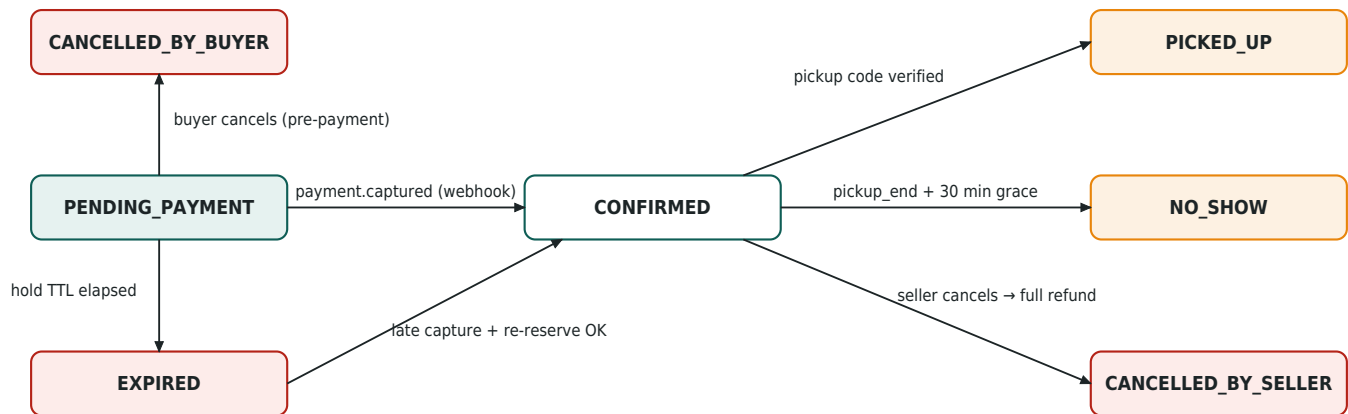
10.4 State machines

Lot



Terminal states: ENDED, CANCELLED. Every transition is a single guarded UPDATE ... WHERE status = <expected> (compare-and-set).

Order



EXPIRED with a late capture that cannot be re-reserved stays EXPIRED and triggers an automatic full refund (payment → REFUNDED).

Payment, refund and store transitions

Entity	From	To	Trigger
Payment	CREATED	CAPTURED	payment.captured webhook / reconciliation
Payment	CREATED	FAILED	payment.failed (attempt-level; order stays pending while hold valid)
Payment	CAPTURED	REFUND_PENDING	refund row created
Payment	REFUND_PENDING	REFUNDED / PARTIALLY_REFUNDED	refund.processed; amount_refunded updated
Refund	PENDING → PROCESSING	SUCCEEDED FAILED	API call accepted → refund.processed / refund.failed; FAILED retried → PENDING until 8 attempts
Store	PENDING_REVIEW	APPROVED REJECTED	Admin action
Store	APPROVED	SUSPENDED	Admin action or 3rd strike (same transaction as strike insert)
Store	SUSPENDED	APPROVED	Admin action only

Transition implementation pattern (every entity): `UPDATE t SET status = :to, version = version + 1, ... WHERE id = :id AND status = ANY(:allowedFrom) RETURNING *`. Zero rows returned means the transition lost a race or is illegal; the service reloads the row and returns a typed error or the idempotent result. Allowed transitions are declared once in `domain/<entity>.transitions.ts` and unit-tested exhaustively (every from x to pair).

10.5 Core algorithms

A1. Reserve (POST /orders)

orders/application/reserve.service.ts (pseudo-code)

```

async reserve(user, dto: {lotId, qty, expectedUnitPrice}, idemKey): Promise<ReserveResult> {
  return idempotency.run(user.id, 'POST /orders', idemKey, dto, async () => {
    assertEmailVerified(user); // BR-BUY-08
    const txResult = await db.transaction(async (tx) => {
      await tx.execute(sql`SELECT pg_advisory_xact_lock(hashtext(${user.id} || ${dto.lotId}))`);
      await policy.assertBuyerLimits(tx, user.id, dto); // BR-BUY-05/06/07, self-purchase
      const lot = await tx.execute(sql`
        UPDATE lots SET qty_available = qty_available - ${dto.qty},
                        status = CASE WHEN qty_available - ${dto.qty} = 0 THEN 'SOLD_OUT' ELSE status END,
                        version = version + 1, updated_at = now()
        WHERE id = ${dto.lotId} AND status = 'LIVE'
        AND now() >= starts_at AND now() < ends_at
        AND qty_available >= ${dto.qty} AND ${dto.qty} <= max_qty_per_order
        RETURNING *, now() AS db_now`);
      if (!lot) throw await explainReserveFailure(tx, dto); // LOT_NOT_LIVE | INSUFFICIENT_STOCK | QTY_LIMIT
      const unitPrice = priceAt(lot, lot.db_now.getTime());
      if (unitPrice > dto.expectedUnitPrice) throw new PriceMismatch(unitPrice); // rolls back decrement
      const order = await orders.insert(tx, { ..., unitPrice, holdExpiresAt: lot.db_now + 5min });
      await outbox.add(tx, 'lot.stock_changed', lot); // version included
      await outbox.add(tx, 'order.reserved', order);
      return { order, lot };
    });
    // outside the DB transaction: external call
    const rzp = await razorpay.createOrder({ amount: order.totalAmount, receipt: order.id },
                                           { timeoutMs: 4000, retries: 2 });
    await payments.insertCreated(order.id, rzp.id, order.totalAmount);
    queue.add('expire-hold', { orderId: order.id }, { delay: 5 * 60_000, jobId: `expire:${order.id}` });
    return toReserveResponse(order, rzp);
  });
}
// If createOrder ultimately fails: cancel the order (compare-and-set), return units, respond 503
// PAYMENT_PROVIDER_UNAVAILABLE. The idempotency record stores this failure so a retry re-attempts cleanly.

```

A2. Webhook intake and processing**payments webhook (pseudo-code)**

```

POST /api/v1/payments/webhook/razorpay (raw body parser, no auth, IP not trusted)
1. verify HMAC-SHA256(rawBody, WEBHOOK_SECRET) == header X-Razorpay-Signature (timingSafeEqual)
   else 400 + metric webhook_signature_invalid_total
2. INSERT INTO payment_events (provider_event_id, event_type, payload)
   ON CONFLICT (provider_event_id) DO NOTHING
3. enqueue 'process-payment-event' {eventId} (jobId = eventId) → respond 200 (< 200 ms)

worker: process-payment-event
payment.captured:
  tx:
    p = SELECT ... FROM payments WHERE provider_order_id = $1 FOR UPDATE
    if !p -> throw Retryable (EC-15)
    if p.status = 'CAPTURED' and p.provider_payment_id <> event.payment_id
      -> create refund DUPLICATE_CAPTURE (EC-17); done
    if event.amount <> p.amount -> flag PAYMENT_MISMATCH, refund AMOUNT_MISMATCH (EC-16); done
    UPDATE payments SET status='CAPTURED', provider_payment_id, fee, tax, method, captured_at
    o = UPDATE orders SET status='CONFIRMED', confirmed_at=now(), pickup_code_* ...
      WHERE id = p.order_id AND status = 'PENDING_PAYMENT' RETURNING *
    if !o: lateCapture(tx, p) -- order EXPIRED/CANCELLED meanwhile (EC-04, EC-06)
    else: ledger.post(tx, 'CAPTURE', {orderId}), outbox 'order.confirmed'
    UPDATE payment_events SET processed_at = now()
  lateCapture:
    if order.status = 'EXPIRED' and lot still LIVE and guarded UPDATE qty succeeds
      -> order EXPIRED -> CONFIRMED (same unit price), ledger, outbox
    else -> refund LATE_CAPTURE_NO_STOCK, notify buyer

```

A3. Hold expiry, scheduler and sweepers

jobs (pseudo-code)

```

job expire-hold {orderId}:
  tx: o = UPDATE orders SET status='EXPIRED' WHERE id=$1 AND status='PENDING_PAYMENT'
                                AND hold_expires_at <= now() RETURNING *
  if o: UPDATE lots SET qty_available = qty_available + o.qty,
                        status = CASE WHEN status='SOLD_OUT' AND now() < ends_at
                                THEN 'LIVE' ELSE status END,
                        version = version + 1 WHERE id = o.lot_id
  outbox 'lot.stock_changed', 'order.expired'

job lot-start {lotId} (delay = starts_at - now, jobId = start:{lotId}:{version})
  UPDATE lots SET status='LIVE', version=version+1 WHERE id=$1 AND status='SCHEDULED' AND starts_at <= now()
job lot-end {lotId} (delay = ends_at - now, jobId = end:{lotId}:{version})
  UPDATE lots SET status='ENDED', end_reason = CASE WHEN qty_available=0 THEN 'SOLD_OUT' ELSE 'TIME' END
  WHERE id=$1 AND status IN ('LIVE','SOLD_OUT') AND ends_at <= now()
-- orders still PENDING_PAYMENT keep their hold until hold_expires_at (never shortened)

sweeper (every 60 s, single-flight via Redis lock 'lock:sweeper' with 55 s TTL):
  run the same guarded UPDATES in bulk for: due SCHEDULED lots, due LIVE/SOLD_OUT lots,
  expired holds, no-shows (CONFIRMED AND pickup_end + 30 min <= now()), stuck outbox rows (> 30 s)

```

A4. Outbox relay

platform/outbox (pseudo-code)

```

loop (every 100 ms, or woken by LISTEN outbox_new):
  BEGIN
    rows = SELECT * FROM outbox_events WHERE published_at IS NULL
          ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED
    for r in rows: redis.publish('events', JSON(r))      -- realtime instances subscribe
                  queue.add(consumersFor(r.type), r)    -- notifications, analytics, alerts
    UPDATE outbox_events SET published_at = now() WHERE id = ANY(ids)
  COMMIT
  -- crash after publish but before COMMIT => republished
  -- => consumers de-dup by (type, aggregate_id, version)
retention: delete published rows older than 7 days (nightly)

```

A5. Pickup verification

pickup (pseudo-code)

```

POST /api/v1/stores/:storeId/pickups/verify {code? | qr?} (STAFF or OWNER of :storeId)
rateLimit('pickup:{storeId}:{sessionId}', 5 per 10 min) -- FR-PICK-03
if qr: [orderId, code, mac] = parse(qr); require timingSafeEqual(mac, hmac(orderId.code))
hash = hmac(PICKUP_SECRET, storeId + ':' + code)
o = UPDATE orders SET status='PICKED_UP', picked_up_at=now(), verified_by=:user, version=version+1
  WHERE store_id=:storeId AND pickup_code_hash=:hash AND status='CONFIRMED'
  AND now() BETWEEN (SELECT pickup_start - interval '15 min' FROM lots WHERE id = lot_id)
  AND (SELECT pickup_end + interval '30 min' FROM lots WHERE id = lot_id)
  RETURNING *
if o: ledger 'SETTLEABLE' after dispute window job; outbox 'order.picked_up'; return {order}
else: look up by (storeId, hash) any status -> PICKED_UP => {alreadyVerified:true, at}
      -> other      => PICKUP_WINDOW_CLOSED | ORDER_NOT_FOUND

```

A6. Ledger postings

Event	Debit (+)	Credit (–)
Capture of ₹200, commission 10%	gateway_clearing 20 000	seller_payable:{store} 18 000 · platform_revenue 2 000
Gateway fee ₹4.72 on that payment	gateway_fees 472	gateway_clearing 472
Full refund (seller cancelled)	seller_payable:{store} 18 000 · platform_revenue 2 000	gateway_clearing 20 000
Payout of settleable balance	seller_payable:{store} X	bank_payouts X

Each row is one ledger_transaction whose entries sum to zero; idempotency_key = {kind}:{orderId}:{paymentId|refundId}.

10.6 REST API specification

Base path `/api/v1`. JSON only. Auth via cookies. Every response carries `x-request-id`. Collections use cursor pagination: `?cursor=&limit=` → `{items, nextCursor}`. The full OpenAPI 3.1 document is generated from Zod schemas (`zod-to-openapi`) and published at `/api/docs` in non-production environments.

Method	Path	Auth	Purpose
POST	<code>/auth/signup</code> · <code>/auth/login</code> · <code>/auth/logout</code>	Public / session	Account and session lifecycle
POST	<code>/auth/verify-email</code> · <code>/auth/resend-otp</code>	Session	Email OTP
POST	<code>/auth/refresh</code>	Refresh cookie	Rotate tokens
POST	<code>/auth/password/forgot</code> · <code>/auth/password/reset</code>	Public	Reset flow
GET	<code>/auth/google/start</code> · <code>/auth/google/callback</code>	Public	OIDC (P1)
GET	<code>/me</code> · PATCH <code>/me</code> · DELETE <code>/me</code>	Buyer	Profile, deletion
GET	<code>/me/sessions</code> · DELETE <code>/me/sessions/:id</code>	Buyer	Session management (P1)
GET	<code>/lots/nearby?lat&lng&radiusKm&category&sort&cursor</code>	Public	Discovery
GET	<code>/lots/:id</code>	Public	Lot snapshot incl. version, serverTime
GET	<code>/lots/search?q&lat&lng</code>	Public	Text search (P1)
POST	<code>/orders</code>	Buyer + Idempotency-Key	Reserve (A1)
POST	<code>/orders/:id/cancel</code>	Buyer + Idempotency-Key	Cancel pending
POST	<code>/orders/:id/payment/verify</code>	Buyer	Client-side signature check (UX only)
GET	<code>/orders</code> · <code>/orders/:id</code>	Buyer	My orders, pickup code
POST	<code>/orders/:id/rating</code> · <code>/orders/:id/dispute</code>	Buyer	Trust
POST	<code>/payments/webhook/razorpay</code>	HMAC signature	Webhook intake (A2)
POST	<code>/stores</code> · GET/PATCH <code>/stores/:id</code>	Buyer → Owner	Store onboarding
POST	<code>/stores/:id/members</code> · DELETE <code>/stores/:id/members/:userId</code>	Owner	Staff (P1)
POST	<code>/stores/:id/lots</code> · PATCH <code>/lots/:id</code> (draft only)	Owner	Create / edit draft
POST	<code>/lots/:id/publish</code> · <code>/end</code> · <code>/cancel</code> · <code>/increase-qty</code>	Owner + Idempotency-Key	Lifecycle
POST	<code>/uploads/presign</code>	Owner / Buyer (disputes)	Presigned PUT URL (type + size bound)
GET	<code>/stores/:id/orders?status=CONFIRMED&date=today</code>	Staff / Owner	Counter view
POST	<code>/stores/:id/pickups/verify</code>	Staff / Owner + Idempotency-Key	Pickup (A5)
GET	<code>/stores/:id/analytics?range=7d</code> · <code>/stores/:id/payouts</code>	Owner	Analytics, payouts
POST	<code>/ai/price-suggestion</code>	Owner	Price Advisor (P1)
POST/DELETE	<code>/me/push-subscriptions</code> · <code>/me/follows/:storeId</code> · <code>/me/alerts</code>	Buyer	Notifications (P1)
GET	<code>/me/notifications</code> · POST <code>/me/notifications/read</code>	Buyer	Notification centre
GET/POST	<code>/admin/stores/review</code> · <code>/admin/stores/:id/{approve reject suspend}</code>	Admin + 2FA	Moderation

Method	Path	Auth	Purpose
GET/POST	/admin/disputes · /admin/disputes/:id/resolve	Admin + 2FA	Disputes
GET	/admin/payments/health · /admin/audit-logs · /admin/flags	Admin + 2FA	Operations
GET	/health/live · /health/ready · /metrics	Internal	Probes, Prometheus

Error envelope

```
HTTP 409
{ "error": { "code": "INSUFFICIENT_STOCK",
  "message": "Only 1 left.",
  "details": { "qtyAvailable": 1, "lotVersion": 42 },
  "requestId": "01J9Z6K3X8..." } }
```

Code	HTTP	Meaning
VALIDATION_FAILED	422	Zod errors in details.fields
UNAUTHENTICATED / FORBIDDEN	401 / 403	No session / not allowed on this resource
EMAIL_NOT_VERIFIED	403	BR-BUY-08
STORE_NOT_APPROVED	403	Store cannot publish
LOT_NOT_LIVE	409	Not started, ended, cancelled or sold out
INSUFFICIENT_STOCK	409	qty > qty_available
QTY_LIMIT_EXCEEDED	409	BR-BUY-05
PENDING_ORDER_LIMIT	409	BR-BUY-06
RESERVATION_COOLDOWN	429	BR-BUY-07, details.until
PRICE_MISMATCH	409	BR-BUY-03, details.currentUnitPrice
SELF_PURCHASE_FORBIDDEN	403	Buyer is a member of the store
HOLD_EXPIRED	409	Order no longer payable
INVALID_TRANSITION	409	State change not allowed from current state
IDEMPOTENCY_KEY_REUSED	422	Same key, different body
IDEMPOTENCY_IN_PROGRESS	409	Same key still executing; retry after Retry-After
PICKUP_WINDOW_CLOSED / ORDER_NOT_FOUND	409 / 404	Pickup verification failures
DISPUTE_WINDOW_CLOSED	422	FR-TRUST-02
RATE_LIMITED	429	With Retry-After header
PAYMENT_PROVIDER_UNAVAILABLE	503	Razorpay failed after retries; reservation released

10.7 Socket.io event contract

Namespace /. Handshake auth: { } (cookie is sent automatically; guests connect anonymously with a signed guest id). All payloads validated by shared Zod schemas; unknown events are dropped and counted.

Direction	Event	Payload	Notes
C → S	time.sync	{ t0 } → ack { t0, serverTime }	Every 5 min and on reconnect
C → S	geo.subscribe	{ lat, lng } → ack { cells: string[] }	Server computes geohash5 + neighbours; rate-limited
C → S	lot.watch / lot.unwatch	{ lotId } → ack { watchers }	Max 5 watched lots per socket
C → S	store.subscribe	{ storeId }	Membership checked
S → C	lot.published	{ lot: LotSummary, serverTime }	geo + store rooms

Direction	Event	Payload	Notes
S → C	lot.started	{ lotId, version, startsAt }	geo + lot rooms
S → C	lot.stock_changed	{ lotId, qtyAvailable, status, version }	Coalesced ≤ 4/s per lot
S → C	lot.ended	{ lotId, reason, version }	reason: TIME SOLD_OUT EARLY CANCELLED MODERATION
S → C	lot.watchers	{ lotId, count }	lot room, throttled 1/5 s (P1)
S → C	order.updated	{ orderId, status, version }	user room; store room for sellers
S → C	notification.created	{ id, type, title, body, unreadCount }	user room
S → C	auth.expired	{}	Client refreshes token then emits auth.refresh
S → C	server.draining	{ reconnectInMs }	Sent on SIGTERM; client reconnects elsewhere with jitter

Shared event schema example

```
// packages/shared/src/events.ts
export const LotStockChanged = z.object({
  lotId: z.string().uuid(),
  qtyAvailable: z.number().int().min(0),
  status: z.enum(['LIVE', 'SOLD_OUT']),
  version: z.number().int().positive(),
});
export type LotStockChanged = z.infer<typeof LotStockChanged>;
```

10.8 Redis key design

Key pattern	Type	TTL	Use
rl:{scope}:{id}	sorted set	window	Sliding-window rate limits
idem:lock:{userId}:{route}:{key}	string (NX)	30 s	In-flight idempotent request lock (row is the durable record)
cache:nearby:{geohash6}:{radius}:{filter sHash}	string (JSON)	3 s	Discovery cache
cache:lot:{id}:{version}	string (JSON)	60 s	Lot snapshot
cache:store:{id}	hash	5 min	Store card + rating
watchers:lot:{id}	HyperLogLog / set	1 h	Watcher count
lock:sweeper · lock:reconcile	string (NX)	55 s / 4 min	Single-flight cron jobs
coalesce:lot:{id}	string	250 ms	Stock event coalescing window
bull:{queue}:*	BullMQ	managed	Queues
socket.io#*	pub/sub	—	Adapter channels

10.9 Queues and jobs

Queue	Job	Trigger	Retries / backoff	Idempotency
lots	lot-start, lot-end	Publish / end-time change	5 × exp 2 s	jobId includes version; guarded UPDATE
orders	expire-hold, mark-no-show	Reservation, confirmation	5 × exp 2 s	Guarded UPDATE
payments	process-payment-event	Webhook intake	10 × exp 5 s (max 10 min)	Unique provider event id
payments	create-refund	Refund row inserted	8 × exp 30 s	Our refund id in notes
payments	reconcile (repeat 5 min)	Cron	—	Redis single-flight lock

Queue	Job	Trigger	Retries / backoff	Idempotency
notify	send-email, send-push, fanout-alerts	Outbox consumers	5 × exp 10 s	(userId, type, entityId) unique
media	process-image	Upload complete	3 × exp 5 s	Output key derived from input key
ai	price-suggestion	API request (awaited ≤ 3.5 s)	0 (fallback instead)	Request id
analytics	lot-result, daily-rollup	lot.ended / nightly	3 × exp 1 min	Upsert by (lotId) / (storeId, day)
platform	integrity-check (10 min), outbox-retention (daily)	Cron	—	Read-only / delete by age

10.10 Configuration (environment variables)

```

NODE_ENV=production          PORT=8080  # entry chosen by start script: api | realtime | worker
DATABASE_URL=...             DATABASE_REPLICA_URL=... DB_POOL_MAX=10
REDIS_URL=...                 PUBLIC_WEB_ORIGIN=https://surplusdrop.app
JWT_ACCESS_SECRET / JWT_REFRESH_SECRET (32+ bytes)    COOKIE_DOMAIN=.surplusdrop.app
PICKUP_CODE_SECRET QR_SIGNING_SECRET DATA_ENCRYPTION_KEY (AES-256-GCM, rotated via key id)
RAZORPAY_KEY_ID RAZORPAY_KEY_SECRET RAZORPAY_WEBHOOK_SECRET
S3_ENDPOINT S3_BUCKET S3_ACCESS_KEY S3_SECRET_KEY CDN_BASE_URL
RESEND_API_KEY EMAIL_FROM VAPID_PUBLIC_KEY VAPID_PRIVATE_KEY
LLM_API_KEY LLM_MODEL LLM_TIMEOUT_MS=3000
OTEL_EXPORTER_OTLP_ENDPOINT SENTRY_DSN LOG_LEVEL=info
# Validated at boot with a Zod schema; the process refuses to start on any missing/invalid value.

```

11. UI / UX Specification

11.1 Design principles

- **Time is the hero.** Price, next-drop countdown and quantity left are the most prominent elements on every lot surface.
- **Honest urgency.** Countdowns and stock are always real; no fake scarcity, no dark patterns. The non-refundable policy is stated before payment.
- **Thumb-first for buyers, glanceable for sellers.** Buyer flows are one-handed on a 360 px phone; the seller counter view is readable at arm's length.
- **Never optimistic about money.** Stock and order state shown after a purchase attempt always come from the server; buttons show a pending state instead.

11.2 Design tokens

Token group	Values
Colour (light)	brand-700 #0F5E56 · brand-50 #E6F2F0 · accent-500 #E8850C (price drops, urgency) · success #15803D · danger #B42318 · ink #1C2426 · muted #5B6B6E · surface #FFFFFF · canvas #F6F8F8
Colour (dark)	canvas #0E1414 · surface #151D1D · brand-300 #5EC2B5 · accent-400 #F5A445 · ink #E7EEEE. Every pair checked for $\geq 4.5:1$ contrast.
Typography	Inter (variable, self-hosted via @fontsource-variable/inter) with tabular numerals for prices and timers. Scale: 12 / 14 / 16 (body) / 20 / 24 / 32 / 40 px.
Spacing & radius	4 px base grid (4, 8, 12, 16, 24, 32, 48). Radius: 8 (inputs), 12 (cards), 20 (sheets), full (chips, pins).
Elevation	3 levels: card, sheet, modal. Dark mode uses lighter surfaces instead of shadows.
Motion	Durations 120 / 200 / 320 ms, easing cubic-bezier(0.2, 0, 0, 1). Price drop: number rolls down + 600 ms accent flash. Disabled under reduced-motion.

11.3 Signature components

Component	Behaviour
LivePrice	Renders <code>priceAt(lot, now + offset)</code> every second via one shared ticker (not one interval per card). Shows current price, struck-through MRP and discount badge. Rolls on drop.
DropCountdown	Circular ring counting down to <code>nextDropAt</code> ; shows "Floor price" when no further drops.
StockBar	Remaining / total with segmented bar; turns accent at $\leq 20\%$, pulses once when changed by a live event.
PriceCurveChart	Step chart of the full schedule with "now" marker; seller version shows sales as dots on the curve.
MapPricePin	Pill-shaped marker showing current price; clusters show count and lowest price.
HoldTimer	Sticky banner on checkout: "Reserved for 4:59". Turns danger at 60 s. At 0 disables payment.
PickupCodeCard	Large 6-digit code (monospace, grouped 3+3) and QR, pickup window, store address with directions link, brightness hint.
CounterScanner	Seller full-screen camera QR scanner with manual code keypad fallback, haptic + colour confirmation.

11.4 Screen inventory

Area	Screen (route)	Key content	Required states
Buyer	Home / Explore (/)	Location chip, radius, category chips, map $\neq$ list toggle, "ending soon" rail	Loading skeleton, empty area, location denied, offline
Buyer	Lot detail (/lots/[id])	Gallery, LivePrice, DropCountdown, StockBar, curve, store card, pickup window, qty stepper, Buy CTA	Scheduled, live, sold out, ended, cancelled

Area	Screen (route)	Key content	Required states
Buyer	Checkout sheet	Summary, HoldTimer, policy consent, Pay (Razorpay)	Reserving, price changed, hold expired, payment failed, confirming
Buyer	Order success	PickupCodeCard, add-to-calendar, push opt-in prompt	Confirming (webhook pending) → confirmed
Buyer	My orders (/orders)	Active and past tabs, rating prompt, dispute entry	Empty, error
Buyer	Store page (/s/[slug])	Profile, rating, live and upcoming lots, follow	Holiday mode, suspended (hidden)
Buyer	Notifications, Settings	Centre, push toggle, alerts, quiet hours, sessions, delete account	—
Auth	Sign up / Log in / OTP / Reset	Minimal forms, inline validation	Locked, expired OTP
Seller	Onboarding wizard (/seller/onboarding)	4 steps: store, location pin, compliance, review	Pending review, rejected (reason)
Seller	Today (/seller)	Live lots with real-time sales, today's pickups, quick "New lot"	No lots, suspended banner
Seller	New / edit lot (/seller/lots/new)	Form + live PriceCurveChart + AI "Suggest pricing"	AI loading / fallback, validation errors
Seller	Counter mode (/seller/counter)	CounterScanner, today's confirmed orders list	Offline queued, already verified, wrong store
Seller	Analytics, Payouts, Staff, Store settings	Charts, tables, invites	Empty periods
Admin	Review queue, Disputes, Users/Stores, Payments health, Flags, Audit log	Dense data tables with filters and detail drawers	SLA overdue, mismatch alerts

11.5 Responsive layout rules

Breakpoint	Buyer	Seller / Admin
< 640 px (mobile)	Bottom tab bar (Explore, Orders, Alerts, Profile); map full-screen with draggable list sheet; checkout as bottom sheet	Bottom tab bar (Today, Counter, Lots, More); forms single column
640-1023 px (tablet)	Two-column list grid; map and list side by side in landscape	Collapsible side rail; counter mode optimised for a tablet on the counter
≥ 1024 px (desktop)	Map left 60%, list right 40%, sticky filters; checkout as centred dialog	Fixed sidebar navigation, data tables with column controls, 12-column grid, max width 1440 px

11.6 Interaction rules (strict)

- Buy button shows Reserving... spinner and is disabled until the server responds; it is re-enabled only after a typed error is rendered.
- Every error code in the API has one human message and one next action in apps/web/lib/errors.ts (for example PRICE_MISMATCH → "Price just changed to ₹48 — buy at ₹48?").
- Skeletons for anything loading longer than 150 ms; no layout shift when data arrives (reserve space).
- Offline: show a persistent banner; lists keep last data greyed with "last updated" time; Buy is disabled offline.
- If the socket is disconnected for more than 10 s, the client falls back to polling the snapshot every 10 s and shows a subtle "live updates paused" chip.
- Toasts only for background events (new order for sellers, refund issued); never for form errors (inline instead).
- Destructive seller actions (cancel lot with orders) require typing the lot title to confirm.

12. Quality: Testing, Load and Observability

12.1 Test strategy

Level	Tooling	Scope	Gate
Unit + property	Vitest, fast-check	pricing.ts, transition tables, policies, heuristics, formatters	100% branch coverage on domain/ and packages/shared
Integration	Vitest, Supertest, Testcontainers	Every endpoint, repositories, jobs, webhook processing with recorded Razorpay payloads, outbox	≥ 85% line coverage on services; all EC-* tests green
Concurrency	Custom harness (Promise.all of N real HTTP calls against Testcontainers)	EC-01...EC-07 with N = 200	Zero oversell across 50 repeated runs
Contract	Shared Zod schemas; OpenAPI diff in CI	API and socket payloads	Breaking change fails CI unless versioned
E2E	Playwright (Pixel 7 + desktop Chrome + WebKit)	Signup → buy → webhook → pickup; seller create lot; admin approve; dispute	Required on main branch
Accessibility	axe-core in Playwright, manual screen-reader pass	All primary screens	No serious/critical violations
Security	ZAP baseline, Gitleaks, dependency audit	Staging	No high findings open at launch

12.2 Load and resilience test plan

Scenario	Setup	Pass criteria
LT-1 Browse	k6: 1 000 VUs hitting nearby + lot detail for 15 min	p95 ≤ 120 ms, errors ≤ 0.1%
LT-2 Socket soak	Artillery: ramp to 10 000 sockets over 5 min across 2 realtime instances, subscribe to geo cells, hold 30 min	CPU ≤ 70%, memory stable (no leak > 5%), event p95 ≤ 500 ms
LT-3 Flash sale	2 000 buyers reserve the same 50-unit lot within 10 s	Exactly 50 units reserved; p95 ≤ 250 ms; 0 5xx
LT-4 Webhook storm	Replay 5 000 webhook events with 30% duplicates and random order	Each processed once; ledger balanced; lag ≤ 30 s
CH-1 Kill realtime instance	During LT-2, kill one instance	All clients reconnected ≤ 5 s p95; no lost stock updates after snapshot refetch
CH-2 Worker outage	Stop workers for 5 min during live lots	Sweepers restore all transitions within 60 s of restart; no missed hold expiry
CH-3 Redis restart	Restart Redis during LT-1	API stays up (rate limiter fails open, logged); realtime clients fall back to polling then recover
CH-4 Razorpay timeout	Inject 5 s latency on createOrder	Reservations released, 503 returned, no orphan holds

Publish the results (graphs, numbers, what broke, what you changed) in `docs/load-testing.md`. This single document is the strongest evidence of seniority in the whole project.

12.3 Dashboards

- **Golden signals:** request rate, error rate, latency per route; socket connections; queue depth and job latency per queue.
- **Money health:** captures/min, refunds pending, failed refunds, webhook lag, reconciliation mismatches, ledger imbalance (must be flat zero).
- **Marketplace:** live lots, reservations/min by result code, hold-expiry rate, conversion, sell-through today.

12.4 Alerts

Alert	Condition	Severity
Oversell detected	oversell_violations_total increases	Critical — page
Unbalanced ledger / orphan capture	Integrity job finds any row	Critical — page
Webhook backlog	Unprocessed payment_events older than 5 min > 0	High
Refund stuck	Refund PENDING/FAILED older than 1 h	High
API error rate	5xx > 1% for 5 min	High
Realtime latency	Fan-out p95 > 1 s for 10 min	Medium
Outbox lag	Oldest unpublished row > 30 s	Medium
Certificate / domain expiry	< 14 days	Low

12.5 Runbooks (in docs/runbooks)

Required before launch, each with symptoms, dashboards to open, diagnosis queries and step-by-step recovery: webhook backlog; oversell alarm; stuck refunds; Redis unavailable; database failover / restore from PITR; Razorpay outage (enable `maintenance_mode` for checkout only); bad deploy rollback; leaked secret rotation.

13. Delivery, DevOps and Release

13.1 Source control and reviews

- Trunk-based development: short-lived branches (≤ 2 days), PRs into main, squash merge, Conventional Commits.
- PR template requires: linked requirement IDs, test evidence, screenshots for UI, migration notes, rollback plan.
- Branch protection: all CI checks green, no direct pushes, signed commits.

13.2 CI/CD pipeline

.github/workflows (outline)

```
on: pull_request
  install (pnpm, cached) → lint → typecheck → unit → integration (Testcontainers)
  → build web + server → e2e (Playwright vs docker-compose stack) → OpenAPI diff → gitleaks + audit

on: push to main
  all of the above → build & scan images (Trivy) → push to registry (tag = git sha)
  → run migrations job on staging → deploy staging (rolling) → smoke tests → notify
  → manual approval → migrations on prod → deploy prod (rolling, 1 instance at a time)
  → post-deploy smoke + error-rate watch 10 min → auto-rollback if 5xx > 2%

scheduled (weekly): load-test.yml against the AWS showcase stack (LT-1...LT-3), results artifact
```

13.3 Database migrations

- drizzle-kit generates SQL migrations; every migration is reviewed as SQL and committed.
- **Expand → migrate → contract**: a release may only add columns/tables or make backward-compatible changes; destructive changes ship one release later.
- Migrations run as a separate one-off job before the new code rolls out; large backfills run as throttled worker jobs, never inside the migration.
- Every migration has a tested down path or a documented forward-fix.

13.4 Environments and data

- pnpm dev starts the full local stack via docker-compose with seeded demo data: 3 stores in Gurugram, 15 lots on staggered schedules, test buyers.
- A demo:simulate script drives synthetic buyers against staging so the public demo always shows live activity (clearly labelled "demo data").
- Production uses Razorpay test mode until business KYC is complete; the UI shows a "test payments" ribbon in that case.

13.5 Definition of Done (every feature)

- Requirement IDs implemented and referenced in code comments where non-obvious.
- Unit + integration tests, including relevant EC-* cases; coverage gates pass.
- API and socket contracts updated in packages/shared; OpenAPI regenerated.
- Metrics, logs and traces added for the new path; alert added if it touches money.
- Responsive at 360 / 768 / 1280 px, dark mode, keyboard and screen-reader checked.
- ADR written for any significant decision; docs and runbooks updated.

13.6 20-week delivery plan

Weeks	Milestone	Deliverables	Exit criteria
1-2	Foundations	Monorepo, CI, docker-compose, Express entry points (api / realtime / worker), Drizzle schema v1, logger/OTel, design tokens, auth (signup, OTP, login, refresh rotation)	CI green; auth integration tests; deployed "hello" to staging
3-4	Stores & lots	Seller onboarding, admin approval, lot CRUD with all BR-AUC rules, image pipeline, shared pricing with property tests	Seller can publish a scheduled lot on staging
5-6	Scheduler & discovery	BullMQ jobs + sweeper, nearby API with PostGIS, explore map/list, lot detail with LivePrice	Lots go live/end on time; explore works on phone
7-8	Realtime	Gateway, rooms, time sync, outbox relay, versioned client cache, reconnect logic	Two browsers see stock change $\leq$ 500 ms
9-10	Orders & payments	Reserve (A1), Razorpay checkout, webhook (A2), expiry, late capture, refunds, ledger	EC-01...EC-20 green; end-to-end purchase in test mode
11-12	Pickup & MVP hardening	Pickup code/QR, counter mode, no-show, ratings, disputes, admin queues. MVP freeze.	All P0 done; Playwright happy paths green
13-14	Notifications & trust	Web Push, follows, saved alerts, quiet hours, strikes, reports, cooldowns	Push works on Android and installed iOS PWA
15-16	Seller analytics & AI	Rollups, dashboard, payout report, Price Advisor with fallback and quota	Analytics reconcile to ledger; AI fallback tested
17-18	Scale & resilience	AWS stack via Terraform, LT-1...LT-4, CH-1...CH-4, fixes, docs/load-testing.md	All pass criteria met and documented
19-20	Polish & launch	Accessibility and performance pass, security review, runbooks, restore drill, onboard 3-5 real stores, demo video, README	Launch checklist complete; real orders completed

14. Master Prompt for AI-Assisted Development

This is the root `CLAUDE.md` of the monorepo. Claude Code reads it at the start of every session, so it encodes the invariants of this SRS and keeps generated code consistent across hundreds of sessions. It imports `project-context.md` (the living project memory) and is complemented by nested `CLAUDE.md` files in `apps/web`, `apps/server`, `packages/shared` and `packages/db`. The skills, agents, hooks and MCP servers it refers to are documented in the companion **SurplusDrop AI Engineering Workflow** guide.

AI assistance speeds you up, but you must be able to explain every line in an interview. Treat generated code like a junior colleague's PR: read it, test it, and rewrite what you do not understand.

CLAUDE.md — master prompt

```
# SurplusDrop – Project Instructions for Claude

You are a senior staff engineer pairing with one engineer on SurplusDrop, a production-grade, real-time Dutch-auction marketplace for near-expiry surplus food in India (pickup only).

The engineer is early in their career and is using this project to learn how professional teams work. Explain the "why" behind non-obvious decisions in one or two lines. Never hand over code they cannot explain.

@project-context.md

## Sources of truth (in priority order)
1. `docs/SurplusDrop_Requirements_Specification.pdf` (SRS v1.1): business rules BR-*, features FR-*, non-functional NFR-*, edge cases EC-*.
2. This file and the nested `CLAUDE.md` files in `apps/*` and `packages/*`.
3. `DESIGN.md` for all visual decisions.
4. `docs/adr/` for recorded decisions.

When they disagree with your instincts, follow them. When they are silent, choose the option that (1) never loses or double-counts money, (2) never oversells stock, (3) keeps the server authoritative – in that order – and propose an ADR with `/adr`.

## Product in one paragraph
Sellers list specific items as live Dutch auctions: the price starts high and drops by a fixed amount every fixed interval down to a floor. Buyers nearby watch prices fall on a map in real time, reserve at the current price, pay via Razorpay within a 5-minute hold, and collect in person during a pickup window using a 6-digit code or signed QR. Pickup only (delivery is a future add-on; keep `fulfilment_type` = PICKUP). Prepaid only. INR only.

## Monorepo map
...
apps/web          React + Vite + React Router (buyer PWA, /seller, /admin)
apps/server       Express + TypeScript; entry points api.ts | realtime.ts | worker.ts
packages/shared   Zod schemas, DTOs, error codes, socket event contracts, pricing.ts
packages/db       Drizzle schema, migrations, seed
packages/ui-tokens design tokens used by Tailwind
packages/config   eslint / tsconfig / prettier presets
infra/           docker-compose, Dockerfiles, terraform, k6
docs/            SRS, adr/, runbooks/, postmortems/, load-testing.md
.claude/         settings.json (hooks, permissions), hooks/, skills/, agents/
...

Package manager: pnpm workspaces + Turborepo. Never use npm or yarn. Add dependencies to the specific workspace: `pnpm --filter @surplusdrop/server add <pkg>`.

## Commands
| Purpose | Command |
|---|---|
| Start everything locally | `pnpm dev` (docker-compose must be up: `pnpm infra:up`) |
```

```

| Lint / format | `pnpm lint` · `pnpm format` |
| Type check | `pnpm typecheck` |
| Unit tests | `pnpm test:unit` |
| Integration tests (Testcontainers) | `pnpm test:integration` |
| E2E (Playwright) | `pnpm test:e2e` |
| DB migration | `pnpm db:generate` then `pnpm db:migrate` |
| Seed demo data | `pnpm db:seed` |
Run a single workspace with `--filter`, e.g. `pnpm --filter @surplusdrop/web test:unit`.

## Fixed stack (do not substitute without being asked)
- TypeScript strict everywhere (`strict`, `noUncheckedIndexedAccess`). No `any`, no `@ts-ignore`.
- Web: React, Vite, React Router, Tailwind, shadcn/ui, TanStack Query, Zustand, React Hook Form + Zod,
  MapLibre GL, Recharts, Motion, Serwist (PWA), socket.io-client.
- Server: Node.js LTS, Express, Zod, Drizzle ORM (raw SQL expected for guarded updates and PostGIS),
  PostgreSQL 16 + PostGIS, Redis 7, BullMQ, Socket.io + redis adapter, Pino, OpenTelemetry.
- Integrations: Razorpay (Orders, Checkout, Webhooks, Refunds), Resend, web-push, Cloudflare R2 + Sharp.
- Tests: Vitest, fast-check, Supertest, Testcontainers, Playwright, k6 / Artillery.

## Non-negotiable invariants
1. Money is integer paaise. Never floats. Format money only in the UI layer.
2. Stock changes only through one guarded UPDATE
  (`... SET qty_available = qty_available - $q WHERE id=$id AND status='LIVE' AND now() >= starts_at
  AND now() < ends_at AND qty_available >= $q RETURNING *`). Never read-then-write. No stock in Redis.
3. Price = `priceAt(lot, dbNow)` from `packages/shared/src/pricing.ts`, using the database clock
  inside the transaction. Reject with PRICE_MISMATCH if server price > `expectedUnitPrice`.
4. Every state transition is compare-and-set (`WHERE id=$1 AND status = ANY($allowed)`); allowed
  transitions live in `domain/<entity>.transitions.ts`.
5. Every money-affecting POST requires an `Idempotency-Key` and uses `platform/idempotency`.
6. The verified Razorpay webhook confirms payment, never the browser callback. HMAC on raw body,
  `timingSafeEqual`, store event (unique id), ack < 200 ms, process in the worker, idempotently.
7. Domain events go to `outbox_events` in the same transaction as the change. Never emit to
  sockets or Redis from a request handler.
8. Every time-based transition has a delayed BullMQ job and is covered by the 60 s sweeper.
9. Every money movement posts a balanced double-entry ledger transaction (entries sum to 0).
10. External calls happen outside DB transactions, with timeouts, retries with jitter; email/push/LLM
    failures never fail an order.
11. The LLM never produces prices; the deterministic heuristic does.

## Server structure rules (Express)
Each module in `apps/server/src/modules/<module>`:
`<m>.routes.ts` (URLs + middleware only) → `<m>.controller.ts` (parse with Zod, call service, send
response) → `<m>.service.ts` (business rules, transactions, outbox) → `<m>.repository.ts` (SQL only);
`domain/` holds pure functions (clock injected); `<m>.schemas.ts` re-exports from `packages/shared`.
- A module never imports another module's repository; call its service or react to its events.
- Errors: throw `AppError(code, details)`; codes only from `packages/shared/src/errors.ts`.
  The error middleware renders `{ error: { code, message, details, requestId } }`.
- IDs UUIDv7. Time `timestampz` UTC; display Asia/Kolkata.
- Config from env vars validated with Zod at boot; exit on invalid config.

```

```
## Web structure rules (React + Vite)
- Mobile-first, responsive at 360 / 768 / 1280 px, light + dark, WCAG 2.1 AA, reduced motion.
- Server state only in TanStack Query; socket events update caches with version checks.
- One shared 1-second ticker drives all LivePrice/Countdown components.
- Never optimistic UI for reservations, payments, cancellations or pickups.
- Every screen has loading (skeleton), empty, error and offline states.
- Every API error code maps to a message + next action in `src/lib/errors.ts`.
- Styling only via tokens from DESIGN.md / `packages/ui-tokens`. No inline styles, no magic numbers.

## How to work on every task
1. Restate the task and list the SRS IDs it covers.
2. List assumptions and questions. If a question touches money, stock or security, STOP and ask.
3. Propose the plan (files, migrations, events, error codes) and wait for approval on non-trivial work.
4. Implement in small steps. Complete files, no placeholders.
5. Tests alongside code; every relevant EC-* gets a test whose name contains the ID.
6. Add logs (no PII), metrics and trace spans; add an alert if it touches money.
7. Finish with: how to run, how to test, migration and rollback notes.
8. Run `/update-context` at the end of any session that changed decisions or status.

## Available skills and agents
Skills: /new-feature /new-endpoint /new-migration /write-tests /local-check /feature-test /review
/edge-case-audit /ui-from-stitch /adr /postmortem /pr /update-context
Agents: code-reviewer, security-reviewer, money-reviewer, test-writer, qa-tester, ui-reviewer

## Git conventions
- Branch per issue: `feat/<short-name>`, `fix/<short-name>`, `chore/<short-name>`.
- Conventional Commits: `feat(orders): reserve lot with guarded update (FR-ORD-01)`.
- Never commit to `main` directly. Never force-push. Never commit secrets.

## Never
- Store card/UPI data or plain-text passwords; put secrets in code or in `.mcp.json`.
- Log email, phone, tokens, pickup codes or raw webhook payloads.
- Use floats for money, `Date.now()` for server business time, or SELECT-then-UPDATE for stock/status.
- Skip or weaken a failing test or a CHECK constraint to make code pass.
- Add a dependency without saying why and what it replaces.
- Change an API or socket contract silently; update `packages/shared` and version it.
```

14.1 Per-task prompt template

Use one task per session

```
TASK: <one sentence, e.g. "Implement POST /orders (reserve) end to end">
SRS IDS: <e.g. FR-ORD-01, BR-BUY-01..09, EC-01..04, EC-07..09, NFR-PERF-02>
CONTEXT: <files that already exist / paste relevant schema or interfaces>
CONSTRAINTS: <anything special for this task>
DELIVERABLES:
- migration (if any) + Drizzle schema update
- shared Zod schemas / error codes / events
- service + repository + controller (+ gateway/job if relevant)
- unit, integration and EC-named tests
- metrics/logs/traces
- short README section: run, test, rollback
Follow the SurplusDrop master prompt strictly. Ask before deciding anything that affects money,
stock or security.
```

15. Future Roadmap (Post-Launch Add-ons)

These features are **out of scope for v1** but the v1 design deliberately leaves room for them. Nothing here may be built before the P1 launch criteria are met.

15.1 Delivery-ready design (applies now)

Decision	What v1 does	Why it matters later
Fulfilment type on every order	<code>orders.fulfilment_type</code> is always PICKUP (CHECK constraint)	DELIVERY becomes a new allowed value, not a new order model
Charges separated from item price	<code>total_amount = qty × unit_price + extra_charges</code> , with <code>extra_charges = 0</code>	Delivery fee slots in without touching payment, refund or ledger logic
Pickup isolated in its own module	<code>pickup</code> module owns codes, verification and no-show	A delivery module can sit beside it with its own state machine
Price and charges locked together	Unit price locked at reservation (BR-BUY-02)	Delivery fee will be quoted and locked in the same transaction

15.2 Add-on 1 — AI assistant for buyers and sellers

- **Buyers:** answers questions about how auctions, holds, pickup and refunds work; finds deals by conversation ("veg meals under ₹100 ending in the next hour"); shows pickup code and window; guides dispute creation.
- **Sellers:** explains platform usage; drafts a lot from a sentence and fills the form for confirmation; answers performance questions from analytics; gives pricing advice via the Price Advisor.
- **Approach:** retrieval over help and policy documents (RAG) plus tool calls into the platform's own read APIs (search lots, my orders, store stats). Every action that changes state or money requires an explicit user confirmation in the UI.
- **Fine-tuning:** only after months of real, consented, anonymised conversations exist; target a smaller, cheaper model for common intents. The retrieval + tools design remains the source of truth for facts.
- **Safety:** prompt-injection hardening, no PII in prompts beyond what the task needs, per-user quotas, and an evaluation set of real questions run in CI before model or prompt changes.

15.3 Add-on 2 — Delivery

Area	Features
Buyer	Delivery or pickup choice at checkout, saved addresses and serviceability check, distance-based fee shown before payment, live partner tracking and ETA, delivery OTP at the door, separate delivery rating
Delivery partner app	Onboarding with documents and vehicle, online/offline toggle, accept/reject offers with earnings shown, navigation to shop then buyer, earnings and payouts
Dispatch engine	Nearest-available partner matching with timed offers and fallback, zone model, batching of orders going the same way, surge pricing (rain, late night, demand)
Shop	"Partner arriving in N min" signal, per-item delivery eligibility
Admin	Partner approvals, live partner map, failed-delivery handling, partner payouts
Policies to define	Minimum order value for delivery, maximum radius and time for perishables, no-partner-available rule (wait / switch to pickup / refund), failed-delivery liability

Rollout: phase 1 integrates a third-party delivery provider's API; phase 2 builds the own partner network, matching and tracking.

16. Appendices

16.1 Appendix A — Glossary

Term	Meaning
Lot	One auction listing; a specific item with a quantity, price schedule and pickup window
Dutch auction	Descending-price auction; each unit sells to the first buyer who accepts the current price
Hold	Units reserved for a PENDING_PAYMENT order for 5 minutes
Floor price	Lowest price the lot can reach
Sell-through	Units sold ÷ units listed
Outbox	Table of domain events written in the same transaction as the change they describe, then relayed
Sweeper	Periodic job that applies any due time-based transition missed by delayed jobs
Late capture	Payment captured after its order's hold expired
Strike	Penalty against a store for seller-caused cancellations or upheld disputes
Geohash5	Geographic cell of roughly 4.9 km × 4.9 km used for realtime rooms

16.2 Appendix B — Price Advisor heuristic (deterministic)

```

input: category, mrp, qtyTotal, minutesToBestBefore (M), history30d {sellThrough, medianSelloutFraction}
urgency = M > 360 ? 0.80 : M > 120 ? 0.70 : 0.60 // start as fraction of MRP
if history.sellThrough < 0.50 urgency -= 0.05
if history.sellThrough > 0.90 && medianSellout < 0.25 urgency += 0.05
start = roundTo(mrp * clamp(urgency, 0.50, 0.90), 500) // nearest ₹5
floorPct = category in (BAKERY, MEALS) ? 0.30 : 0.40
floor = min(roundTo(mrp * floorPct, 100), floor(start * 0.9))
duration = clamp(M - 60, 30, 180) minutes // leave ≥ 1 h before best-before
interval = duration >= 90 ? 300 : 120 seconds
steps = max(1, floor(duration * 60 / interval) - 1) // reach floor one step before end
decrement = max(100, ceil((start - floor) / steps / 100) * 100) // whole rupees
output = validate against BR-AUC-01.08; on violation, fall back to seller's current values

```

The LLM receives these numbers and the seller's title as quoted data, and returns { explanation: string (≤ 60 words), improvedTitle?: string (≤ 80), improvedDescription?: string (≤ 500) }, validated by Zod.

16.3 Appendix C — Launch checklist

Area	Check
Correctness	All P0 + P1 requirements done; all EC-* tests green; integrity job silent for 7 days on staging
Payments	Webhook secret rotated for prod; reconciliation verified with a deliberately dropped webhook; refunds tested for every reason
Security	OWASP Top 10 checklist signed off; ZAP baseline clean; admin 2FA enforced; secrets only in secret store
Reliability	Restore drill done and timed; runbooks written; alerts routed and tested
Performance	LT-1...LT-4 and CH-1...CH-4 passed and documented; Lighthouse ≥ 90 on mobile
Legal & policy	Terms, privacy notice (DPDP Act), refund policy page, seller agreement, FSSAI self-declaration shown
Product	3–5 real stores onboarded; 20+ real completed orders; demo video (2–3 min) recorded

16.4 Appendix D — Evidence to show recruiters

- A live URL with demo buyer and seller accounts, plus a 2–3 minute video of two phones watching a price drop and one buying.
- README with the architecture diagram, ADR list, and a "hard problems I solved" section (oversell, late capture, reconnect consistency).
- docs/load-testing.md with graphs and the story of what broke and how you fixed it.
- One written postmortem of a real bug in industry format (impact, timeline, root cause, fix, prevention).
- Resume line with numbers, for example: "Built a real-time auction marketplace (React, Express, Socket.io, PostgreSQL/PostGIS, Redis) sustaining 10k concurrent WebSockets and zero oversell across 2 000-request flash sales; Razorpay payments with webhook reconciliation and a double-entry ledger."